Baiyuan GEO Platform — Technical Whitepaper

Engineering Practice for Brand Visibility in the Generative AI Era

Vincent Lin (Baiyuan Technology)

2026-04-18

Abstract

This whitepaper documents the engineering practice of Baiyuan Technology in developing "Baiyuan GEO Platform" (2024–2026), a SaaS system targeting Generative Engine Optimization (GEO). The platform helps brands be cited accurately and consistently in responses from generative AI services (ChatGPT, Claude, Gemini, Perplexity, and others). The book presents a seven-dimension citation-rate scoring algorithm, an AI-Bot-friendly shadow document delivery mechanism (AXP), a Schema.org three-layer entity knowledge graph, and a closed-loop hallucination detection and auto-remediation system. Published under CC BY-NC 4.0.

Contents

Chapter 1 — The Era of Generative Engine Optimization **5**
Table of Contents . . . 5
1.1 Generative search has taken over user habit . . . 5
1.2 AI Citation Rate: a brand-new and life-or-death metric . . . 6
1.3 GEO is not an extension of SEO — it is an independent discipline . . . 6
1.4 Market landscape: first movers abroad, whitespace elsewhere . . . 7
1.5 Why this book, and how to read it . . . 8
Key takeaways . . . 8
References . . . 8

Chapter 2 — Baiyuan GEO Platform: System Overview **9**
Table of Contents . . . 9
2.1 Design philosophy: from monitoring tool to closed-loop system . . . 9
2.2 Three modules . . . 9
2.3 Core data flow . . . 10
2.4 Technology stack . . . 11
2.5 Multi-tenant data isolation . . . 11
2.6 AI platform coverage . . . 12
Key takeaways . . . 12
References . . . 12

Chapter 3 — The Seven-Dimension GEO Scoring Algorithm **13**
Table of Contents . . . 13
3.1 Why a single score is not enough . . . 13
3.2 Seven-dimension design . . . 13
3.3 Weight philosophy . . . 15
3.4 Function skeleton . . . 16
3.5 Compared to a traditional SEO score . . . 16
3.6 Limits of the algorithm . . . 16
Key takeaways . . . 17

References . . . 17

Chapter 4 — Stale Carry-Forward: Engineering for Signal Continuity 18
Table of Contents . . . 18
4.1 Problem statement . . . 18
4.2 Why the intuitive solutions are all wrong . . . 18
4.3 Stale Carry-Forward design . . . 19
4.4 Design trade-offs . . . 20
4.5 The philosophy of data freshness . . . 20
4.6 Function skeleton . . . 21
4.7 Applicability to other domains . . . 22
Key takeaways . . . 22
References . . . 22

Chapter 5 — Multi-Provider AI Routing: Fault Tolerance as Starting Architecture 23
Table of Contents . . . 23
5.1 Structural risks of single-vendor dependency . . . 23
5.2 modelRouter architecture . . . 23
5.3 Three engineering challenges . . . 24
5.4 The retry-stacking trap . . . 25
5.5 Switch triggers and observability . . . 26
5.6 Fallback path health management . . . 27
5.7 Function skeleton . . . 28
Key takeaways . . . 29
References . . . 29

Chapter 6 — AXP Shadow Documents: Serving Clean Content to AI Bots via Cloudflare Workers 30
Table of Contents . . . 30
6.1 Why one HTML cannot serve both . . . 30
6.2 AXP: the structure of a shadow document . . . 31
6.3 Cloudflare Worker injection . . . 32
6.4 AI bot UA list and detection strategy . . . 33
6.5 Path conflicts for SaaS self-brands . . . 34
6.6 Automatic sitemap generation . . . 35
6.7 JSON-LD flattening pitfalls . . . 35
6.8 GSC indexing field notes . . . 36
6.9 RAG knowledge base integration: per-brand KB automation . . . 36
Key takeaways . . . 38
References . . . 38

Chapter 7 — Schema.org Phase 1: 25 Industries × Three-Layer @id Interlinking 39
Table of Contents . . . 39
7.1 Schema.org's role has shifted in the AI era . . . 39
7.2 Industry-specialized @type across 25 categories . . . 39
7.3 Three-layer @id interlinking . . . 41
7.4 Physical vs online: divergent field weights . . . 42
7.5 Data completeness algorithm . . . 43
7.6 Dual entry points: Wizard + Edit . . . 44
7.7 GBP URL Parser . . . 44
7.8 Function skeleton . . . 45
Key takeaways . . . 46
References . . . 46

Chapter 8 — GBP API Integration Strategy: One-Way Sync from Source of Truth to Exter-

nal Output **47**
Table of Contents . . . 47
8.1 Why GBP is the source of truth for physical businesses . . . 47
8.2 GBP API approval gate and process . . . 47
8.3 Hosting model choice . . . 48
8.4 Field-mapping table . . . 49
8.5 Sync frequency and quota . . . 50
8.6 Remedy for no-webhook . . . 51
8.7 Phase 1–4 roadmap . . . 51
Key takeaways . . . 52
References . . . 52

Chapter 9 — Closed-Loop Hallucination Detection and Auto-Remediation **53**
Table of Contents . . . 53
9.1 Why detection alone is insufficient . . . 53
9.2 Five types of AI hallucination . . . 53
9.3 Primary detection: NLI classification + ChainPoll . . . 54
9.4 Central shared RAG: SaaS infrastructure . . . 56
9.5 L1 LLM Wiki: an active semantic layer . . . 57
9.6 Remediation: ClaimReview generation and multi-path injection . . . 59
9.7 Two-tier rescan loop . . . 60
9.8 Convergence timing and acceptance . . . 61
Key takeaways . . . 61
References . . . 62

Chapter 10 — Phase Baseline Testing: Longitudinal AI-Perception Comparison with Fixed Question Sets **63**
Table of Contents . . . 63
10.1 Regular scan's longitudinal blind spot . . . 63
10.2 Phase baseline testing design . . . 64
10.3 Data-path isolation from regular scanning . . . 64
10.4 Four observation axes . . . 65
10.5 Operational guidance . . . 66
Key takeaways . . . 66
References . . . 67

Chapter 11 — Five-Brand Field Observations: Six Weeks of Anonymized Data **68**
Table of Contents . . . 68
11.1 Brand portraits (anonymized) . . . 68
11.2 GEO score distribution . . . 69
11.3 Platform coverage asymmetry . . . 69
11.4 Schema.org completeness and citation rate . . . 70
11.5 AXP deployment before/after . . . 70
11.6 Customer-side pitfalls . . . 71
11.7 Three unexpected findings . . . 71
11.8 First-month commercial validation . . . 72
Key takeaways . . . 73
References . . . 73

Chapter 12 — Limitations, Open Problems, and Future Work **74**
Table of Contents . . . 74
12.1 What the platform cannot do . . . 74
12.2 Unpredictability of AI model version shifts . . . 75
12.3 Open research problems . . . 76
12.4 Roadmap . . . 76

12.5 An invitation to practitioners and researchers . . . 77
Key takeaways . . . 77
References . . . 78

Appendix A — Glossary (Full) **79**
A . . . 79
B . . . 79
C . . . 79
D . . . 79
E . . . 80
F . . . 80
G . . . 80
H . . . 80
I . . . 80
J . . . 80
K . . . 80
L . . . 80
M . . . 81
N . . . 81
O . . . 81
P . . . 81
Q . . . 81
R . . . 81
S . . . 81
T . . . 82
U . . . 82
W . . . 82
Standards cross-reference . . . 82

Appendix B — Public API Specification (Excerpt) **83**
Common conventions . . . 83
1. Brands . . . 83
2. Scans . . . 84
3. Entity (Schema.org brand entity) . . . 85
4. AXP (shadow document) . . . 86
5. Hallucinations . . . 86
6. Baseline (Phase baseline testing) . . . 86
7. Webhooks (customer-receivable) . . . 87
Error codes . . . 87

Appendix C — References **88**
1. Official specifications and standards . . . 88
2. Technical frameworks and development tools . . . 88
3. Industry research and statistics . . . 89
4. Books and monographs . . . 89
5. Related academic work (selected) . . . 89
How to cite this book . . . 90

Appendix D — Figure Catalog and Production Guidelines **91**
D.1 Complete catalog . . . 91
D.2 Production style guide . . . 92
D.3 Production status . . . 94

Chapter 1 — The Era of Generative Engine Optimization

> Users no longer *search* for answers. They *ask* for them. When retrieval is replaced by generation, the rules of brand visibility have to be rewritten.

Table of Contents

- 1.1 Generative search has taken over user habit
- 1.2 AI Citation Rate: a brand-new and life-or-death metric
- 1.3 GEO is not an extension of SEO — it is an independent discipline
- 1.4 Market landscape: first movers abroad, whitespace elsewhere
- 1.5 Why this book, and how to read it
- Key takeaways
- References

1.1 Generative search has taken over user habit

Before 2023, the sentence *"I need to look something up"* was effectively interchangeable with *"I'll open Google."* By 2024 that equivalence was cracking; by late 2025 it had stopped being true.

The numbers tell a compact story:

- **ChatGPT** processes more than **2.5 billion user queries per day** (OpenAI Q3 2025 disclosure).[1]
- **Perplexity** reports **780 million monthly queries**, with year-over-year growth in the triple digits.[2]
- **Google AI Overview**, rolled out fully from May 2024, has driven click-through rates on the traditional blue-link SERP down by an average of **34–48%** across measured verticals, per SimilarWeb and the Digital Research Index; in high-information domains (medical, legal, SaaS selection) the drop is closer to **60%**.[3]

The conclusion for any brand is a single sentence:

> The *entry point* through which users find you is migrating from *ten blue links* to *a paragraph of AI-generated prose*. If your brand does not appear in that paragraph, you effectively do not exist in that user's decision path.

This is not a forecast. It is already the world we ship into.

Fig 1-1: Information channel migration (illustrative)

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Information Retrieval Channel Share (2023-2025, illustrative)"
    x-axis ["2023 Q1", "2023 Q4", "2024 Q2", "2024 Q4", "2025 Q2", "2025 Q4"]
    y-axis "Share of Queries (%)" 0 --> 100
    bar [85, 78, 65, 52, 42, 35]
    line [5, 10, 18, 28, 36, 42]
```

Fig 1-1: Bars represent "traditional Google SERP"; the line represents "generative AI (including AI Overview)". Figures vary by region and research firm; shown as a trend, not an authoritative series.

---

[1] OpenAI. (2025). *Usage & revenue update, Q3 2025.* Official quarterly disclosure.
[2] Perplexity AI. (2025). *Year in review 2024: search volume & engagement.* Official blog.
[3] SimilarWeb. (2025). *The state of generative search: AI Overview impact on publisher traffic.* Research report.

1.2 AI Citation Rate: a brand-new and life-or-death metric

When a user asks ChatGPT, Claude, Perplexity, Copilot, or Gemini questions like:

- "What are the best B2B marketing-automation tools?"
- "Which dermatology clinics in downtown Chicago have the best reviews?"
- "Which CRM SaaS fits a 50-person company?"

the AI generates a paragraph **containing specific brand names**. Brands that are named enter the user's shortlist. Brands that are not named **do not surface at all in the conversation**. Users rarely follow up with *"are there others?"* — the same way they rarely paged past the first ten Google results in 2015.

We quantify this phenomenon as **AI Citation Rate**: the fraction of representative intent queries in which the brand is proactively mentioned by the AI. It is not a click-through rate, not an impression count, not a ranking. It is, plainly, *whether the AI remembers you and names you*.

Its properties are unlike any traditional SEO metric:

Property	Traditional SEO	AI Citation Rate
Transparency	Rule-based signals (PageRank, Core Web Vitals)	Black box; no published rules
Cross-platform	Google dominates; most signals portable	Divergent — ChatGPT, Claude, DeepSeek, Kimi each behave differently
Time stability	Algorithm updates ~quarterly	Model retraining can shift behavior weekly
Output form	A clickable link	A natural-language sentence (good or bad)

The implication: in the AI era, being *linked to* matters less. Being *described* matters more. Whether the description is accurate, flattering, and complete becomes a brand asset in its own right.

1.3 GEO is not an extension of SEO — it is an independent discipline

If we need a single-sentence boundary: **SEO is about getting Google to rank you first; GEO is about getting the AI to mention you at all.**

Fig 1-2: SEO vs GEO core differences

Dimension	SEO	GEO
Success outcome	A clickable blue link	A brand name inside a natural-language paragraph
Trigger mechanism	Keyword match + authority signals + UX signals	"Entity-association strength" inside model training and retrieval augmentation
Operable levers	Content, backlinks, structured data, Core Web Vitals	Structured entities, trusted sources, hallucination remediation, AI-bot crawling compatibility

Dimension	SEO	GEO
Success metrics	Rank, CTR, time-on-site	Citation rate, position quality, narrative sentiment, cross-platform consistency
Time horizon	Weeks to months	Model retraining cycle (typically quarterly)
Primary audience	Human browsers	AI models + their crawlers and retrieval pipelines

Fig 1-2: SEO and GEO are parallel disciplines, not successive ones. Treating GEO as a sub-topic of SEO leads to systematic resource misallocation.

Many SEO practitioners claim: *"Do SEO well and AI will naturally cite you."* That was partially true in 2023 and is effectively false by 2025. The reasons are twofold:

First, the AI's training data is no longer a projection of Google's index. Major LLMs train on Common Crawl, licensed publishers, open knowledge bases (Wikipedia, Wikidata), and vendor-curated retrieval corpora. Google rankings are a small and indirect input.

Second, structured data matters far more to AI than to traditional search. An AI interprets an entity through Schema.org JSON-LD, Wikidata triples, and knowledge-graph linkages — not through H1/H2 keyword density. A site with a perfect SEO score but no Schema.org structure looks almost blank to an AI.

GEO is therefore not SEO's *next version*. It is a parallel discipline with different inputs, different levers, and different failure modes. Any investment plan that treats them as continuous will misfire.

1.4 Market landscape: first movers abroad, whitespace elsewhere

Outside the English-language market the tooling landscape was effectively empty at the time of writing. A handful of 2024-era GEO tools emerged in the US and Europe:

- **Profound** (USA) — focused on Fortune 500, cross-platform brand monitoring and content optimization
- **Otterly.ai** (Europe) — SMB-oriented, AI citation dashboards with competitor comparisons
- **AthenaHQ** (Israel) — enterprise-focused, RAG integration and content auditing

These are strong products. Each, however, centers on the English-speaking market: their interfaces, their AI-platform coverage, and their underlying knowledge-graph corpora all assume the USA and Western Europe as center of gravity. Their coverage of Traditional Chinese content, Taiwan-native brand knowledge, and Chinese-origin AI models (Baidu Ernie, DeepSeek, Moonshot Kimi, Zhipu ChatGLM) was — and largely remains — absent.

Taiwan and the wider Traditional-Chinese market in the same period had effectively *no native GEO tooling*. Most brand owners and marketing agencies were still doing "Google-search-your-brand-and-count-positions" manual verification; occasionally asking ChatGPT *"what are the best X brands"* and noting the result by hand. A visibly widening market gap.

1.5 Why this book, and how to read it

Baiyuan GEO Platform was built to close that gap. From a 2024 prototype to the production service shipping in 2026, we accumulated enough engineering experience that it became worth recording in whitepaper form: the algorithms, the architectures, the fault-tolerance patterns, the AI-bot-friendly content delivery design, the structured-entity management, the automated hallucination remediation, and the data-governance practices that keep all of it running in a multi-tenant SaaS.

This book is **not a product brochure**. It is **not a user manual**. It is an **engineering report**: why we made each design choice, which roads we took that turned out to be dead ends, and which patterns we believe other teams can reuse.

The intended readers are three groups:

1. **B2B decision makers** — to establish a mental model for *AI-era brand visibility* and avoid the category error of treating GEO as an SEO sub-topic
2. **Engineering leaders and architects** — to borrow patterns like multi-AI-provider fault tolerance, signal continuity, and closed-loop automated remediation
3. **Developers and practitioners** — to see how Schema.org, Cloudflare Workers, pgvector, BullMQ, and similar tools compose into a real, shipping product

The remaining 11 chapters cover system overview, core algorithms, external-visibility construction, quality-assurance loops, and finally real-world observations. Customer-specific PII and commercially sensitive numbers are always presented as aggregates or anonymized; algorithmic details are disclosed in skeleton form while specific weights are intentionally withheld — the balance we chose between knowledge sharing and commercial reality.

Key takeaways

- Generative AI search has substantively changed the way users obtain information; traditional SERP click-through rates are down 34–60% in high-information verticals
- *AI Citation Rate* is the core metric of GEO; it differs from SEO ranking in nature, mechanism, and operable levers
- GEO is parallel to SEO, not successive — treating it as a sub-topic guarantees resource misallocation
- First movers (Profound, Otterly, AthenaHQ) serve the English-speaking market; Chinese-language markets remain under-served
- This book is the engineering report behind Baiyuan GEO Platform, written for decision makers, engineering leaders, and developers

References

Chapter 2 — Baiyuan GEO Platform: System Overview

Baiyuan GEO Platform is the subject of this book, operated by Baiyuan Technology. This chapter is the map index the remaining ten chapters extend.

Table of Contents

- 2.1 Design philosophy: from monitoring tool to closed-loop system
- 2.2 Three modules
- 2.3 Core data flow
- 2.4 Technology stack
- 2.5 Multi-tenant data isolation
- 2.6 AI platform coverage
- Key takeaways
- References

2.1 Design philosophy: from monitoring tool to closed-loop system

First-generation GEO tools tend to stop at the *monitoring* layer: a dashboard that tells the user *"here is your score."* Monitoring is diagnosis, not treatment. Customers who see a low score immediately ask: *"So what do I do next?"*

The starting point of Baiyuan GEO Platform was to treat the entire flow as a **closed loop**:

Fig 2-1: Six stages of the closed-loop system

```
flowchart LR
    A[Scan] --> B[Diagnose]
    B --> C[Optimize]
    C --> D[Deploy]
    D --> E[Verify]
    E --> A
    E -.->|hallucination| F[Remediate]
    F --> D
```

Fig 2-1: Every stage must be automatable, quantifiable, and auditable. Any stage that requires manual intervention once is a scar on the system.

From this starting point we partitioned the platform into three modules.

2.2 Three modules

2.2.1 Scan Module — monitoring

Responsibility: **periodically send brand questions to AI, capture the responses, structure them, and score them.**

Core duties:

- Intent-query generation — produce 20–60 representative questions per brand per industry
- Multi-platform dispatch — distribute the same question set to 15+ AI platforms and search engines
- Response extraction — from natural-language answers, extract *was the brand mentioned*, *at what position*, *with what tone*, *alongside which competitors*
- Score computation — combine seven dimensions into the GEO total (see Ch 3)

- Signal continuity — activate Stale Carry-Forward when a platform outage occurs (see Ch 4)

Technology choices:

- Task queue: **BullMQ** on Redis — retries, priorities, rate limits
- AI routing: home-grown **modelRouter** with a primary OpenAI-compatible aggregator and per-vendor direct fallbacks (see Ch 5)
- Scan modes: regular daily + sentinel 4-hour (see Ch 9) + Phase baseline weekly/bi-weekly (see Ch 10)

2.2.2 Visibility Module — external visibility

Responsibility: **help AI recognize and cite the brand more easily.**

Core duties:

- **Structured entity management** — Schema.org JSON-LD, 25 industry-specialized `@type` × three-layer `@id` interlinking (see Ch 7)
- **AXP shadow documents** — produce pure-HTML + JSON-LD + Markdown "clean content" versions for AI bots, decoupled from the human-facing site (see Ch 6)
- **Cloudflare Worker injection** — at the network edge, detect AI-bot User-Agents and dynamically return the shadow document (or pass through to the origin)
- **GBP integration** — treat Google Business Profile as the source of truth for physical-location brands; one-way sync into Schema.org (see Ch 8)
- **Knowledge-source construction** — link to Wikipedia, Wikidata, LinkedIn, and other authoritative platforms that AI crawlers consume

2.2.3 Quality Module — quality assurance

Responsibility: **detect and remediate AI's incorrect beliefs about the brand**, so that the loop converges.

Core duties:

- **Hallucination detection** — extract brand claims from AI responses and run NLI + ChainPoll against combined knowledge sources
- **ClaimReview generation** — for each confirmed hallucination, emit a Schema.org `ClaimReview` node
- **Knowledge-base sync** — push corrections into the RAG system so subsequent retrieval covers the error
- **Two-tier rescan loop**:
  - Tier 1 sentinel (4h / search-type AI) — rapid verification of whether the remediation was picked up
  - Tier 2 full (24h / knowledge-type AI) — deeper confirmation with score + fingerprint comparison

---

2.3 Core data flow

Fig 2-2: A full brand governance cycle

```
flowchart TD
    U[User creates brand] --> W[Wizard guides Schema.org fill-in]
    W --> G[System emits AXP shadow doc + JSON-LD]
    G --> CF[CF Worker serves AI bots with shadow content]
    CF --> S[Scan Module dispatches intent queries<br/>to 15+ AI platforms]
    S --> R[Responses flow back → scored]
    R --> D[Dashboard shows 7 dimensions<br/>+ competitor comparison + gap analysis]
    R --> Q[Quality Module detects hallucinations]
    Q --> C[Emit ClaimReview<br/>inject into AXP + RAG]
```

```
    C --> V[Sentinel / full rescan verifies]
    V -->|converged| S
    V -.->|still present| C
```

Fig 2-2: The loop can run on the order of hours (sentinel) or days (full scan). Users see only the score trend in the dashboard; all eight steps above are happening continuously beneath.

2.4 Technology stack

Layer	Technology	Primary role
Edge	Cloudflare Workers	AI-bot UA detection, shadow-document injection, sitemap / robots management
Frontend	Next.js 16 (Webpack) + React 19 + TypeScript + Tailwind v4	Dashboard, brand management, Wizard, bilingual i18n (zh-TW / en)
API	Node.js + Express 4 + Helmet + express-rate-limit + Zod	REST API, multi-tenant, JWT + 2FA
Worker	BullMQ 5 + Node.js task processors	Scan, scoring, AXP generation, hallucination detection, RAG sync
AI Routing	Custom modelRouter + OpenAI SDK + vendor-specific SDKs	Multi-provider fault tolerance
Data	PostgreSQL 16 + pgvector + Redis 7	Relational data, vector retrieval, cache, queue
Deploy	Docker Compose on AWS Lightsail (PROD) / local Docker (UAT)	Environment isolation
RAG	Centralized shared RAG engine (internal SaaS infrastructure)	Multi-tenant knowledge-base sync and query

As of 2026-04-18 the system is at v2.19.4; ~20 minor versions shipped, 139 migrations, 60+ frontend pages, 180+ API endpoints.

2.5 Multi-tenant data isolation

Baiyuan GEO Platform is a B2B SaaS serving multiple customers from one database. **Data isolation is a contract-grade obligation**, not a nice-to-have. We use a **double-insurance** strategy:

1. **Row-Level Security (RLS) on PostgreSQL** — RLS policies on `brands`, `scan_results`, `geo_scores` and other core tables filter by `current_setting('app.org_id')`. Even if application-level SQL is written incorrectly, data cannot leak across tenants.
2. **App-level filter** — every query-layer API also carries `org_id` explicitly, forming a second layer of validation. If an RLS policy is inadvertently dropped, the application layer still preserves isolation.

Fig 2-3: Double-insurance illustration

```
flowchart LR
    R[HTTP Request] --> M[Auth Middleware<br/>resolves org_id]
    M --> C[Controller]
    C --> Q[Query Layer<br/>explicit WHERE org_id]
    Q --> PG[(PostgreSQL)]
    M -.->|SET app.org_id| PG
    PG --> RLS{RLS Policy<br/>filters via current_setting}
    RLS --> D[(Data)]
```

Fig 2-3: Application layer and database layer each run their own filter. A single-layer slip cannot cause cross-tenant leakage.

This design came from a near-miss incident early on: relying only on app-level conditions means humans forget; relying only on RLS means policy changes slip through CI unnoticed. Double insurance requires two failures to coincide before any leakage happens.

2.6 AI platform coverage

At time of writing, Baiyuan GEO Platform covers **15 AI platforms**, grouped into three families:

Category	Count	Representative platforms
Global general-purpose	7	OpenAI GPT-4o / 4o-mini, Anthropic Claude, Google Gemini, Meta Llama, Mistral, xAI Grok, Cohere
Chinese-language	5	Baidu Ernie, DeepSeek, Moonshot Kimi, Zhipu ChatGLM, Alibaba Qwen
Search-type	3	Perplexity, ChatGPT Search, Google AI Overview (crawler side)

Adding coverage for one platform is not just *wiring up an API*. It also requires: model-ID mapping, `extraParams` design, timeout / retry tuning, hallucination-detection templates, score calibration, and UI presentation. The all-in cost is typically **2–4 engineering days per platform**.

Key takeaways

- Baiyuan GEO begins from a *closed-loop* stance — monitor, diagnose, optimize, deploy, verify — all automated
- Three modules (Scan / Visibility / Quality) each correspond to later chapters
- The stack: Next.js 16 + Express + BullMQ + PostgreSQL 16 + pgvector + Redis as the core
- Multi-tenant isolation uses RLS + app-level filter as double insurance against single-layer human error
- Coverage of 15 AI platforms (7 global + 5 Chinese + 3 search-type) at ~2–4 eng-days per platform onboarded

References

- Ch 3 — Seven-Dimension Scoring Algorithm
- Ch 4 — Stale Carry-Forward
- Ch 5 — Multi-Provider AI Routing
- Ch 6 — AXP Shadow Document
- Ch 7 — Schema.org Phase 1
- Ch 8 — GBP API Integration
- Ch 9 — Closed-Loop Hallucination Remediation
- Ch 10 — Phase Baseline Testing

Chapter 3 — The Seven-Dimension GEO Scoring Algorithm

> A single number can compress any complex system into something comparable. It can also compress the differences that deserved to be seen.

Table of Contents

- 3.1 Why a single score is not enough
- 3.2 Seven-dimension design
- 3.3 Weight philosophy
- 3.4 Function skeleton
- 3.5 Compared to a traditional SEO score
- 3.6 Limits of the algorithm
- Key takeaways
- References

3.1 Why a single score is not enough

Baiyuan GEO's v1 score in 2024 had exactly one metric: **Citation Rate** = *mentions* ÷ *queries*. Simple, intuitive, comparable. After three months of production use, however, we collected a growing pile of cases where the metric was **misleading because the same total represented very different brand health**.

Two real cases, both scoring 55 in v1, had nothing in common beneath the surface:

- **Brand A**: mentioned in 11 of 20 queries, always in the *final* paragraph in an *"and there's also…"* tone, exclusively on OpenAI and Anthropic.
- **Brand B**: mentioned in 11 of 20 queries, 8 of them in the *first or second sentence*, across 7 different AI platforms, with detailed descriptions.

The underlying "AI awareness" of these two brands was orders of magnitude apart. The score flattened them into the same bucket. A customer looking only at that single number lost every piece of information that could drive action.

The root problem: **GEO is a multi-dimensional phenomenon**. *How often* you are mentioned, *where* in the response, *on how many platforms*, *with what tone*, *in what depth*, *with what cross-platform consistency* — these are independent signals. Averaging them into one number destroys the information they each carry.

3.2 Seven-dimension design

The v2 score decomposes *a brand's state in the AI ecosystem* into **seven orthogonal dimensions**.

Fig 3-1: Seven-dimension radar (v1 vs v2, illustrative)

```
%%{init: {'theme':'base'}}%%
graph LR
    A[Citation Rate] --- B[Position Quality]
    B --- C[Query Coverage]
    C --- D[Platform Breadth]
    D --- E[Sentiment]
    E --- F[Content Depth]
    F --- G[Consistency]
    G --- A
```

Fig 3-1: Adjacency in the diagram is informational, not hierarchical. All seven dimensions are scored independently and composed into a total. An actual per-brand radar chart belongs in the PDF edition with real data.

3.2.1 Citation Rate

Definition: fraction of representative intent queries in which the brand is proactively mentioned (0–100).

Starting point: `mentioned_count / query_count`.

Refinements:

- Deduplicate false positives — competitor name collisions, product-line lookalikes
- Lemma matching — brand full name, abbreviation, English form, mixed-script form all count as the same mention
- Compute per-platform rate first, then weighted-average to the brand total — avoids one noisy platform dominating

Weight: the largest single-dimension weight, but deliberately capped at ~25% (one-quarter) so the score does not silently regress into the v1 single-metric world.

3.2.2 Position Quality

Definition: weighted average of the positions at which the brand appears in AI responses.

Scoring table:

Position	Weight
First sentence / first list item	1.0
Top third	0.8
Middle third	0.5
Last third / mentioned-in-passing	0.2

This dimension answers a question that was common under v1: *"Why does my citation count look healthy but the result doesn't feel useful?"* — typically because the brand is mentioned *late and in passing*.

3.2.3 Query Coverage

Definition: diversity of *query types* in which the brand is mentioned.

Logic: intent queries are categorized ("best-of", "comparison", "problem-solution", "beginner recommendation", etc.). Coverage = *mentioned types ÷ total types*.

What it reveals: some brands only ever surface in *"what are the best X tools"* questions, while being completely absent from *"should I choose A or B"* comparisons. Those are different battlefields requiring different content strategies.

3.2.4 Platform Breadth

Definition: on how many AI platforms the brand is proactively mentioned.

Logic: *platforms with at least one mention ÷ monitored platforms*.

Reveals platform bias. A SaaS that is mentioned on the OpenAI family but never on the Chinese-model family (DeepSeek, Kimi, etc.) is not experiencing random variance — it is showing a specific content-visibility problem rooted in training-data provenance or structured-data coverage.

3.2.5 Sentiment Score

Definition: aggregated tone of each mention.

Logic: an independent sentiment classifier scores each mention as positive / neutral / negative; aggregated to 0–100. We deliberately do **not** use the same AI's reasoning to grade itself; that would create an *"A AI evaluating A AI"* circularity.

This dimension is quiet in normal operation (neutrality typically dominates at 70–80%) but sensitive in crisis: when the neutral share drops below 50%, it almost always correlates with an external negative content surface that can be identified.

3.2.6 Content Depth

Definition: the depth of the description attached to each brand mention.

Logic: measure the length, entity density (how many related facts — product lines, founders, deployment scenarios — appear in the same breath), and syntactic complexity of the text segment about the brand.

This dimension separates "being named" from "being introduced." For B2B SaaS, education brands, and professional services, Content Depth matters *more* than Citation Rate itself. A deep description converts; a name-drop rarely does.

3.2.7 Consistency

Definition: inverse of the standard deviation of the six dimensions above, across different AI platforms (normalized to 0–100).

Logic: a brand with high consistency looks the same on ChatGPT, Claude, Gemini, and DeepSeek. A brand with low consistency has a vivid persona on one platform and a fuzzy shadow on another.

This dimension does not change the other six — it is a *reliability signal*. High consistency means the AI ecosystem has converged to a stable entity picture of the brand; low consistency means divergent recall across platforms.

3.3 Weight philosophy

The seven dimensions are not averaged with equal weight. Weight allocation follows three principles:

1. **Importance** — Citation Rate and Content Depth most directly drive "is the brand known?" — they get higher weight.
2. **Noise sensitivity** — Sentiment and Consistency are vulnerable to single-response outliers — lower weight to prevent jagged scores.
3. **Operability** — Platform Breadth and Query Coverage are the dimensions most directly responsive to *proactive* effort — weight enough that the score *responds* to improvement work.

The book discloses the **skeleton of the formula and the coarse tiering (high / medium / low)** but does not publish precise numeric weights. This is not for secrecy; it is to **prevent customers from optimizing the metric instead of the substance**:

> If you know that a dimension is 30% of the total, you will pour resources into that dimension alone. The score will rise, but the AI's actual perception of the brand will not change. We would rather you focus on *improving overall content quality* than on reverse-engineering weights.

This aligns with how search engines handled their own ranking signals. Google never published PageRank's exact weights for the same reason.

3.4 Function skeleton

```
// Weights are resolved from config; concrete values are deliberately not exposed.
function calcGEOScore(scanResults, brandId) {
  const dims = {
    citation:    computeCitationRate(scanResults, brandId),
    position:    computePositionQuality(scanResults, brandId),
    coverage:    computeQueryCoverage(scanResults, brandId),
    breadth:     computePlatformBreadth(scanResults, brandId),
    sentiment:   computeSentimentScore(scanResults, brandId),
    depth:       computeContentDepth(scanResults, brandId),
    consistency: computeConsistency(scanResults, brandId),
  };

  const weighted =
    dims.citation    * W_CITATION +
    dims.position    * W_POSITION +
    dims.coverage    * W_COVERAGE +
    dims.breadth     * W_BREADTH +
    dims.sentiment   * W_SENTIMENT +
    dims.depth       * W_DEPTH +
    dims.consistency * W_CONSISTENCY;

  return {
    total: Math.round(weighted),
    dimensions: dims,
    version: SCORING_VERSION,
  };
}
```

3.5 Compared to a traditional SEO score

Facet	SEO Score	GEO Score
Inputs	Page content + backlinks + UX metrics	AI response text + entity matching + cross-platform aggregation
Output form	Link ranking	Natural-language mention
Time granularity	Daily	Daily + sentinel 4h + Phase-baseline weekly
Comparability	Cross-site comparable	Not strictly cross-industry comparable (query spaces differ)
Version sensitivity	Low	High (tied to AI model release cycles)

The table is not rhetoric. It is a reminder: **GEO Score is a brand-*state* indicator, not a brand-*quality* ranking.** Cross-industry comparisons of the absolute number are meaningless; only same-industry, same-time-window, same-query-pool comparisons are.

3.6 Limits of the algorithm

An honest list of what this seven-dimension system **cannot** do:

- **Does not reflect business outcomes** — a high GEO score does not equal better conversion. It is a proxy for AI-visibility, not a revenue metric.
- **Sensitive to model retraining** — when OpenAI or Anthropic releases a new major version, scores may shift 3–10 points platform-wide. That is an external shift, not a brand shift.
- **Query space is subjective** — two equally reasonable query sets for the same industry can produce different scores. We mitigate with Phase baseline testing (see Ch 10) but cannot eliminate.
- **Chinese-language coverage is still catching up** — Chinese models' sentiment classifiers and position-detection accuracy lag their English counterparts.

These are not algorithmic failures. They are honest statements about the nature of GEO measurement. Any tool claiming *"precise quantification of AI perception"* should be regarded with suspicion.

Key takeaways

- A single Citation Rate misjudges brands of different character as equivalent when totals happen to match
- Seven dimensions (Citation / Position / Coverage / Breadth / Sentiment / Depth / Consistency) each carry independent signal and use
- Weights follow *importance / noise sensitivity / operability*; numbers deliberately unpublished to avoid the metric-gaming prisoner's dilemma
- GEO Score is a brand-**state** indicator, not a brand-**quality** ranking; cross-industry comparisons are meaningless
- Scores respond to AI model version releases; Phase baseline testing provides longitudinal stabilization

References

- Ch 4 — Stale Carry-Forward: Signal Continuity Design
- Ch 10 — Phase Baseline Testing
- Google Search Central. *How Search works*. https://www.google.com/search/howsearchworks/
- Schema.org. *ClaimReview schema*. https://schema.org/ClaimReview

---

Chapter 4 — Stale Carry-Forward: Engineering for Signal Continuity

> A GEO score reflects *the brand's state in AI perception*, not *the data pipeline's health*. Conflating the two empties the score of meaning.

Table of Contents

- 4.1 Problem statement
- 4.2 Why the intuitive solutions are all wrong
- 4.3 Stale Carry-Forward design
- 4.4 Design trade-offs
- 4.5 The philosophy of data freshness
- 4.6 Function skeleton
- 4.7 Applicability to other domains
- Key takeaways
- References

4.1 Problem statement

Any system that relies on external AI services will at some point experience **partial or full failure**. The sources of failure are many:

- Network-level packet loss, CDN re-routing, DNS jitter
- API-level 5xx, 429 rate limits, token quota exhaustion
- Model-level version retirement, server restart, regional outage
- Account-level card expiry, payment delay, billing system lag

These failures are **daily events** in production engineering, not exotic exceptions. The problem: when they hit the scanning pipeline, **they must not contaminate the GEO score**.

Imagine a scenario. Brand X's Citation Rate has been stable at ~55 over the last 30 days. One morning, 3 of the 6 AI platforms scanning X fail simultaneously. If we count failures as zeros, X's score drops to ~27.5 for the day. A customer opens the dashboard, assumes their brand has suddenly lost favor, and starts a panic response. But the AI's perception of X has not changed at all. Only the data pipeline has.

This is **the first class of error**: treating pipeline failure as brand change.

4.2 Why the intuitive solutions are all wrong

Faced with the above, three intuitions commonly surface. All three have problems.

Option A: dynamic denominator (exclude failed platforms)

Idea: compute the score only over platforms that succeeded this scan.

Problem: a varying denominator makes the score **jitter as a sawtooth**. Today 10 platforms all succeed, score is 60; tomorrow only 6 succeed (including the top-scoring ones), score is 62; the day after, all 10 succeed again, back to 60. The trend line becomes unreadable — the user cannot tell whether a movement is real or an artifact of which platforms happened to be online.

Option B: failure counts as zero

Idea: every failed platform contributes a 0 to the day's average, preserving the denominator.

Problem: this tells the algorithm *"the brand is absent from this platform today"* when the truth is *"we don't know the brand's state on this platform today."* **"Not mentioned" and "not observed" are fundamentally different states.** Conflating them poisons downstream trend analysis, hallucination detection, and competitor comparison.

Option C: silent drop (don't record)

Idea: since the scan failed, pretend this time slice never existed.

Problem: the time-series database develops **gaps**, and the system can no longer distinguish *"weekend idle"* from *"scan failed"* from *"scan ran but produced no change."* Compensating logic added later to paper over this gap grows in complexity fast.

All three share a single root problem: **none of them distinguishes between "data is missing" and "data is zero."**

4.3 Stale Carry-Forward design

Our approach is to **carry forward the most recent successful value from history** and mark the entry explicitly as stale.

Fig 4-1: Three strategies under the same outage (illustrative)

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Score under mid-scan platform outage (illustrative)"
    x-axis ["Day 1", "Day 2", "Day 3 (outage)", "Day 4", "Day 5"]
    y-axis "GEO Score" 0 --> 80
    line [55, 56, 27, 28, 56]
    line [55, 56, 55, 55, 56]
```

Fig 4-1: Top curve — Option B "failures as zero" producing a false crash. Bottom curve — Stale Carry-Forward holding continuity while surfacing `isStale` in the UI.

Algorithm

1. **Detect**: for a given platform in the current scan, failure rate reached **100%** (all queries either got no response or timed out).
2. **Look up history**: walk back up to **200 rows** of that brand × platform scan history and find the most recent **non-zero successful** `sov_score`.
3. **Carry forward**: use that historical value for the day's row, with two marker fields set:
   - `isStale = true`
   - `lastSuccessAt = <historical_timestamp>`
4. **Tell the user, honestly**: the dashboard shows the score with a red badge; hover tooltip reads "☐ out of contact for N hours — showing last successful value."
5. **Downstream still works**: Consistency, trend analysis, and competitor comparison consume the carried value rather than hitting a gap.

Fig 4-2: Platform-data state machine

```
stateDiagram-v2
    [*] --> Fresh
```

```
Fresh --> Stale: current scan 100% failed
Stale --> Fresh: next scan succeeds
Stale --> Expired: stale beyond threshold (72h)
Expired --> Fresh: scan succeeds
Expired --> Reset: still failed after 30d
Reset --> Fresh: rebuild baseline
```

Fig 4-2: Fresh □ Stale is the normal recovery path. Expired and Reset are exception branches reserved for long-running outages.

4.4 Design trade-offs

4.4.1 Why the lookback window is 200 rows, not unbounded

Reason: to avoid carrying forward stale-enough-to-be-wrong values in pathological cases. If a platform stays down for weeks, the most recent "success" could be months old — decoupled from the brand's current state. 200 rows (about 6–7 months at daily cadence) is a soft upper bound that balances *"maintain continuity"* with *"keep data relevant."* Beyond that cap we refuse to carry forward, write `null`, and surface *"data rebuild needed"* in the UI.

4.4.2 What about first-scan brands?

Strategy: for a brand with no history, if a platform fails, we do **not** carry forward. Zero or mean imputation would be fabricated data. Instead the frontend shows *"first scan — data under construction,"* distinguishing *missing history* from *existing history but stale*.

4.4.3 Interaction with Phase baseline testing

Phase baseline testing (see Ch 10) runs on an **independent data path** and is *not* subject to Stale Carry-Forward. The purpose of baseline testing is to measure real longitudinal change; any carried-forward value would contaminate that purpose. If a baseline scan encounters platform failure, that Phase result is marked `status = incomplete` and re-run after recovery.

4.4.4 When does Stale become alarming?

The `isStale` flag is informational, not an alarm. But if the same platform remains stale beyond **72 hours**, the system escalates:

- Frontend badge changes from static red to pulsing red
- Tooltip text changes to "□ *prolonged outage — check configuration"*
- An email notification is dispatched to the brand's account admin

After **7 days** the platform enters `Expired` and the UI states explicitly *"data expired — not included in current scoring."* Carry-forward stops.

4.5 The philosophy of data freshness

The legitimacy of carrying forward rests on an assumption: **AI perception changes on the scale of weeks, not hours.** Model retraining, knowledge-graph updates, and external news cycles all move on weekly-or-slower cadences. Using yesterday's value as a proxy for today's (when today's is unavailable) is **statistically justified**.

But the assumption has limits:

- **High-frequency signals** (social sentiment, Google Trends) — carry-forward beyond 24 hours is already misleading
- **Low-frequency stable signals** (Schema.org completeness, GBP verification state) — carry-forward can tolerate weeks
- **Events-in-flight** (PR crisis, product launch) — the smoothing effect of carry-forward is itself a risk

Our platform restricts carry-forward to **citation-rate-related dimensions** (Citation, Position, Sentiment) and does not apply it to structured-data state, fingerprint comparison, or other indicators.

4.6 Function skeleton

```
// Simplified illustration. Production code handles multi-platform fan-out,
// lookback tuning, and emits observability events.
async function enrichWithStaleCarryForward(platform, brandId, currentResult) {
  const currentFailed = currentResult.successCount === 0;
  if (!currentFailed) {
    return { ...currentResult, isStale: false };
  }

  const lastSuccess = await db.query(`
    SELECT sov_score, position_quality, sentiment, scanned_at
      FROM scan_results
     WHERE brand_id = $1 AND platform = $2
       AND sov_score IS NOT NULL AND sov_score > 0
  ORDER BY scanned_at DESC
     LIMIT 1
    OFFSET 0
  `, [brandId, platform]);

  if (!lastSuccess.rows.length) {
    return { ...currentResult, isStale: false, reason: 'no_baseline' };
  }

  const historical = lastSuccess.rows[0];
  const ageHours = (Date.now() - historical.scanned_at.getTime()) / 3_600_000;

  if (ageHours > MAX_CARRY_FORWARD_HOURS) {
    return { ...currentResult, isStale: true, expired: true };
  }

  return {
    sov_score:        historical.sov_score,
    position_quality: historical.position_quality,
    sentiment:        historical.sentiment,
    isStale:          true,
    lastSuccessAt:    historical.scanned_at,
    staleAgeHours:    ageHours,
  };
}
```

Fig 4-3: Frontend isStale badge (illustrative)

```
OpenAI GPT-4o      55 pts   stale 14h
Anthropic Claude   62 pts
Google Gemini      48 pts   stale 14h
```

Fig 4-3: Red marker and hover tooltip state plainly: "this value is the last successful reading, not the current one." The user cannot be accidentally misled.

4.7 Applicability to other domains

Stale Carry-Forward is not unique to GEO. Any *"high-frequency sampling, unreliable source, continuity-requiring"* signal system can apply it:

Domain	Use case	Parameters to adjust
IoT sensing	Intermittent sensor loss	Lookback in minutes
Financial quotes	Exchange brief disconnects	Not applicable — finance cannot compromise immediacy
Social monitoring	API quota exhausted	Shorter lookback — signal changes fast
Ad attribution	Pixel loss	Combine with probabilistic matching
Supply-chain visibility	EDI transmission interruption	Longer lookback tolerable

The common precondition: **change rate « sampling rate**. When that fails (as in finance), carry-forward is wrong.

Key takeaways

- The core of Stale Carry-Forward is **separating brand state from pipeline health** as two distinct signals
- Three intuitive solutions (dynamic denominator, zero-impute, silent drop) all break time-series readability
- The design has four steps: detect ☐ history lookup ☐ mark ☐ inform honestly in UI. **The UI must not hide staleness.**
- Lookback limit, first-scan handling, and Phase-baseline interaction are the three main trade-offs
- The pattern generalizes to any signal system where *change rate « sampling rate*

References

- Ch 3 — Seven-Dimension GEO Scoring
- Ch 5 — Multi-Provider AI Routing
- Ch 10 — Phase Baseline Testing
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. (Chapter 8, "The Trouble with Distributed Systems," is a general reference for handling partial failure.)

Chapter 5 — Multi-Provider AI Routing: Fault Tolerance as Starting Architecture

> Any system that depends on a single AI vendor is a fragile system. Multi-path fault tolerance belongs at the start of the architecture, not bolted on after an incident.

Table of Contents

- 5.1 Structural risks of single-vendor dependency
- 5.2 modelRouter architecture
- 5.3 Three engineering challenges
- 5.4 The retry-stacking trap
- 5.5 Switch triggers and observability
- 5.6 Fallback path health management
- 5.7 Function skeleton
- Key takeaways
- References

5.1 Structural risks of single-vendor dependency

Before discussing *how* to tolerate faults, it is worth articulating *why*. Single-vendor AI dependency carries five **structural** risks that are independent of the vendor's size or brand:

1. **Shared token bucket** — all customers, all models share a single rate-limit pool. A traffic spike in one customer propagates into a platform-wide degradation.
2. **Model retirement / version switch** — when a vendor retires an old model (e.g., `gpt-4-0314`, `claude-2`), every feature depending on it must migrate immediately.
3. **Regional failure** — a single cloud region outage interrupts every user in that region.
4. **Billing / account issues** — an expired card, a delayed payment, an API-key rotation mistake all produce complete disconnection.
5. **Policy and geopolitical shifts** — a vendor changes service scope, a region changes policy, model terms of use are updated.

These are not *"handle when they happen"* risks. They are *"they will happen"* realities. Depending on any single vendor means locking all five categories into your own SLA. For a SaaS this is unacceptable design.

5.2 modelRouter architecture

Baiyuan GEO uses a **primary/fallback dual-path** abstraction named `modelRouter`.

Fig 5-1: Dual-path routing architecture

```
flowchart LR
    App[Application layer<br/>scan / eval / hallucination] --> MR[modelRouter]
    MR -->|primary| Agg[OpenAI-compatible<br/>aggregator]
    MR -->|fallback| D1[OpenAI direct]
    MR -->|fallback| D2[Anthropic direct]
    MR -->|fallback| D3[Google direct]
    MR -->|fallback| D4[DeepSeek direct]
    MR -->|fallback| D5[Moonshot direct]
    MR -->|fallback| Dn[other vendor-direct]
```

```
    Agg --> LLM1[GPT-4o / Claude / Gemini / ...]
    D1 --> LLM1
```

Fig 5-1: The application layer calls an abstract `modelRouter.complete()`. The decision of which path to take is encapsulated inside the router and is transparent to the application.

Two paths, two trade-offs

Path	Properties	Typical use
Primary (aggregator)	One API endpoint covers multiple vendor models, unified billing, low onboarding cost	Day-to-day scanning, scoring, general inference
Fallback (vendor-direct)	Per-vendor API key, per-vendor billing, per-vendor SDK	Activated on primary failure; also used when a vendor-specific feature is needed

The primary path's strength is *"onboard once, use everywhere."* The fallback path's strength is *independence*. The `modelRouter` abstracts both so the application layer neither knows nor cares which path a given request took.

5.3 Three engineering challenges

Dual-path routing sounds simple. Implementing it surfaces three thorny issues.

5.3.1 Model IDs differ between aggregator and native layers

The most commonly underestimated issue. The same model is identified by different strings in the aggregator and the native API:

Vendor-native ID	Aggregator ID
`deepseek-chat`	`deepseek-v3`
`moonshot-v1-8k`	`kimi-k2`
`claude-3-5-sonnet-20241022`	`claude-3-5-sonnet`
`qwen-plus`	`qwen3-plus`
`meta-llama/Llama-3.3-70B-Instruct`	`llama-3.3-70b`

Without handling this mapping, **a model name that succeeds on the primary path returns HTTP 404 on direct-call fallback**. The fix is a mapping table:

```
const DIRECT_MODEL_ID_MAP = {
  // aggregator ID → direct provider ID
  'deepseek-v3':       { provider: 'deepseek',  model: 'deepseek-chat' },
  'deepseek-r1':       { provider: 'deepseek',  model: 'deepseek-chat' },
  'kimi-k2':           { provider: 'moonshot',  model: 'moonshot-v1-8k' },
  'qwen3-plus':        { provider: 'alibaba',   model: 'qwen-plus' },
  'grok-2':            { provider: 'xai',       model: 'grok-2-latest' },
  'claude-3-5-sonnet': { provider: 'anthropic', model: 'claude-3-5-sonnet-20241022' },
  // ... full mapping covers 15+ entries
};
```

Every model addition or deprecation must update this table or a path silently breaks.

5.3.2 extraParams divergence

Vendor-native APIs each require certain parameters the aggregator silently defaults for you. When switching to direct, those parameters must be supplied explicitly:

Model family	Required `extraParams`
Qwen3 family	`enable_thinking: false` (otherwise returns the reasoning trace, wasting tokens)
DeepSeek-R1	Defaults to reasoning mode; set `temperature: 0.6` for stable output
Claude (Anthropic SDK)	`max_tokens` **mandatory** (OpenAI-style vendors allow it omitted)
Gemini	`safetySettings` defaults are strict; commercial content often needs relaxation

We maintain a `DIRECT_EXTRA_PARAMS` table per provider and merge it in at path-switch time.

5.3.3 Reasoning-model trade-offs

DeepSeek-R1 and **OpenAI o1/o3** are reasoning-type models — they think internally before responding, producing 15–30 second latencies that exceed our scanner's 25-second timeout.

The fix is **not** to raise the timeout (that would drag the entire scan cadence). The fix is to **fall back to the same vendor's non-reasoning variant**:

Primary model	Fallback direct model	Compromise
`deepseek-r1`	`deepseek-chat`	Loses `reasoning_content` field
`o1-mini`	`gpt-4o-mini`	Loses the reasoning chain; citation detection does not depend on it

For GEO scanning, **citation detection only needs the final text**. The reasoning chain is a nice-to-have, not a requirement. Trading `reasoning_content` for completion rate is the right call.

5.4 The retry-stacking trap

This is the most common production trap. A typical system has *at least three layers of retry*:

```
Business Layer
                    withRetry(fn, { retries: 3 })
Router Layer
                    primary fail → fallback (= another attempt)
SDK Layer
                    OpenAI SDK built-in maxRetries: 2
```

Multiplied out: a single OpenAI SDK timeout of 30s × 2 retries × withRetry 3 × primary-to-fallback switch 1 = **worst case 7 minutes** before the system gives up. The scanner expects a request to finish in 25 seconds. Instead, a single request blocks an entire worker for more than an order of magnitude longer.

The correct principle

> Only the outermost layer gets to decide when to give up. Every inner layer has `maxRetries: 0` and an explicitly set `timeout`.

Concrete implementation:

```
// SDK layer: no internal retry
const openai = new OpenAI({
  apiKey: API_KEY,
  maxRetries: 0,
  timeout: 20_000, // 20s hard limit
});

// Router layer: single failover, no loop
async function routeComplete(request) {
  try {
    return await primaryPath(request);
  } catch (err) {
    if (isRetryableError(err)) {
      return await fallbackPath(request);
    }
    throw err;
  }
}

// Business layer: caller decides retry policy
const result = await withRetry(
  () => modelRouter.complete(request),
  { retries: 2, backoff: 'exponential' }
);
```

Three layers each handle their own responsibility. Total worst-case elapsed time = 20s × 2 (primary + fallback) × 3 (outer retry) = 120s. Predictable, monitorable, boundable.

5.5 Switch triggers and observability

Not every error should trigger a fallback. Some errors a switch cannot help — switching wastes a fallback slot for nothing.

Switch decision tree

```
flowchart TD
    E[Primary path error] --> C{Error type}
    C -->|HTTP 5xx| S[switch to fallback]
    C -->|HTTP 429 rate-limit| S
    C -->|Timeout| S
    C -->|Network error| S
    C -->|HTTP 401 auth| X[re-raise immediately<br/>switching won't help]
    C -->|HTTP 400 param| X
    C -->|HTTP 402 payment| X
    C -->|Model not found| S
    S --> R[record event<br/>provider, model, reason, duration]
    X --> R
    R --> O[emit metrics<br/>for observability]
```

Fig 5-2: Switch decision tree. 5xx / 429 / timeout / network errors are transient* and warrant a switch; 401 / 400 / 402 are *permanent* and should not trigger one.*

Required observability fields

Every switch event must record:

Field	Purpose
`provider_primary`	Which vendor served the primary path
`provider_fallback`	Which vendor served the fallback (if triggered)
`model_requested`	The model ID the application asked for
`model_actual`	The model that actually served (may differ on fallback)
`reason`	transient / permanent / timeout / model_not_found / ...
`latency_primary_ms`	Primary path elapsed time (including timeout wait)
`latency_fallback_ms`	Fallback path elapsed time
`status`	success_primary / success_fallback / both_failed

Over time these metrics answer important questions: *which vendor is least stable, which model triggers fallbacks most often, which time windows are riskiest?* Without this data, debugging is driven by customer complaints.

5.6 Fallback path health management

The worst state a fallback can be in is *not* "failing" — it is **"unused daily, broken when actually needed."** Three mechanisms mitigate this.

5.6.1 Startup smoke test

On every worker startup, send a minimal request (`"ping"` or a one-word completion) to **every fallback provider**:

```
async function smokeTestFallbacks() {
  const providers = Object.keys(DIRECT_CLIENTS);
  const results = await Promise.allSettled(
    providers.map(p => pingProvider(p, { timeout: 5_000 }))
  );
  return results.map((r, i) => ({
    provider: providers[i],
    ok: r.status === 'fulfilled',
    error: r.status === 'rejected' ? r.reason.message : null,
  }));
}
```

Output hits the startup log and a `/health/fallbacks` endpoint. Any broken path is visible immediately to ops.

5.6.2 Periodic health check

Every 30 minutes, a light check using a low-cost prompt (`"OK"` □ 2 tokens). Three consecutive failures flip the provider's status to *"fallback unavailable"* in the UI.

5.6.3 Vendors we cannot cover

Some providers have no available fallback. Three common cases:

Case	Example	Policy
Closed-source, no public API	Some proprietary enterprise models	UI states "no fallback"; primary failure ☐ fail
Requires customer-supplied key	Certain paid frontier models	User fills key in settings; left blank ☐ no auto-fallback
Geopolitical restriction	Certain cross-border services	Dynamic enablement by customer's region

Transparency beats *"pretending to have coverage."* Telling the customer *"this provider has no fallback"* is better than letting them discover it during an outage.

5.7 Function skeleton

5.7.1 modelRouter main entry

```
export async function complete({ prompt, model, temperature, ...opts }) {
  const request = buildRequest({ prompt, model, temperature, ...opts });

  try {
    const result = await callPrimary(request);
    emitMetric({ status: 'success_primary', model });
    return result;
  } catch (err) {
    if (!isRetryableError(err)) {
      emitMetric({ status: 'permanent_fail', reason: err.code, model });
      throw err;
    }

    const fallback = resolveFallback(model);
    if (!fallback) {
      emitMetric({ status: 'no_fallback_available', model });
      throw err;
    }

    const fallbackReq = mapRequestForProvider(request, fallback);
    try {
      const result = await callDirect(fallback.provider, fallbackReq);
      emitMetric({
        status: 'success_fallback',
        provider_fallback: fallback.provider,
        model_actual: fallback.model,
      });
      return result;
    } catch (fbErr) {
      emitMetric({ status: 'both_failed', model });
      throw fbErr;
    }
```

```
  }
}
```

5.7.2 resolveFallback

```
function resolveFallback(aggregatorModelId) {
  // 1. Exact match in DIRECT_MODEL_ID_MAP
  if (DIRECT_MODEL_ID_MAP[aggregatorModelId]) {
    return DIRECT_MODEL_ID_MAP[aggregatorModelId];
  }

  // 2. Pattern match for versioned models
  //    e.g. "deepseek-r1-0528" → "deepseek-chat"
  for (const [pattern, target] of DIRECT_PATTERN_MAP) {
    if (pattern.test(aggregatorModelId)) return target;
  }

  // 3. No fallback available
  return null;
}
```

These two functions are the heart of the routing architecture. Token counting, prompt assembly, and response normalization all layer on top of them.

Key takeaways

- Single-vendor dependency is a structural risk, not a "we'll handle it later" item
- `modelRouter` abstracts primary/fallback dual paths; the application layer is blind to the choice
- Three engineering challenges: model-ID mapping, extraParams divergence, reasoning-model timeout compromises
- Retry must be centralized at the outermost layer; SDK and router set `maxRetries: 0` to avoid exponential elapsed time
- Switch decisions depend on error type — transient triggers a switch, permanent is re-raised
- Fallback paths need active health management (smoke test + periodic check + transparency about uncovered vendors)

References

- Ch 2 — System Overview
- Ch 4 — Stale Carry-Forward
- OpenAI. *Node.js SDK retry configuration*. https://github.com/openai/openai-node
- Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf. (Circuit Breaker, Bulkhead, and Timeout patterns are the canonical reference.)

Chapter 6 — AXP Shadow Documents: Serving Clean Content to AI Bots via Cloudflare Workers

> The website for humans and the content for AI should not be the same HTML. Forcing one document to serve both leaves both parties worse off.

Table of Contents

- 6.1 Why one HTML cannot serve both
- 6.2 AXP: the structure of a shadow document
- 6.3 Cloudflare Worker injection
- 6.4 AI bot UA list and detection strategy
- 6.5 Path conflicts for SaaS self-brands
- 6.6 Automatic sitemap generation
- 6.7 JSON-LD flattening pitfalls
- 6.8 GSC indexing field notes
- 6.9 RAG knowledge base integration: per-brand KB automation
- Key takeaways
- References

6.1 Why one HTML cannot serve both

A modern website is designed for humans:

- Client-side rendering (CSR) — content only appears after JavaScript runs
- Dynamic cards, carousels, modals
- Cookie consent banners, ad trackers, A/B-test SDKs
- Deeply nested `<div class="col-md-6">` layouts that lack semantic meaning
- Background video, WebGL, animations

For humans these are UX. For AI crawlers they are noise. When an AI bot (GPTBot, ClaudeBot, PerplexityBot, Googlebot, and 20+ others) fetches a modern brand page, three failure modes are common:

1. **JavaScript fails or times out** — most AI bots do not execute JS or execute it only in restricted form; SPA pages return a bare `<div id="app"></div>`.
2. **Main-content extraction fails** — HTML noise is too dense; the AI cannot distinguish brand information from UI chrome.
3. **Structured data missing** — Schema.org JSON-LD is often injected into dynamic positions the crawler never reaches.

The result: the AI's perception of the brand is either **wrong** or **thin**. The remedy is not to rewrite every customer website for AI's benefit. It is to **prepare a dedicated, clean shadow document *for* the AI**.

Fig 6-1: Two views of the same brand

```
flowchart LR
    Brand[Brand data] --> H[Human version<br/>React / Vue / Next.js<br/>full UI / animation / ads]
    Brand --> A[AI version shadow doc<br/>Pure HTML + JSON-LD<br/>pure semantics, no noise]
    H -->|Browser| User[User experience]
    A -->|AI Bot| LLM[Model training and retrieval]
```

Fig 6-1: The same brand data produces two renderings — human version optimized for experience, AI version optimized for semantics.

6.2 AXP: the structure of a shadow document

AXP (AI-ready eXchange Page) is Baiyuan's name for such shadow documents. Each AXP page has three layers.

Fig 6-2: AXP three-layer structure

```
flowchart TB
    subgraph AXP["AXP shadow document"]
      HTML["(1) Pure HTML semantic skeleton<br/>h1 / h2 / p / ul / table"]
      JSONLD["(2) Schema.org JSON-LD<br/>Organization / Service / Person<br/>three-layer @id interlinkin
      MD["(3) Markdown raw text block<br/>for RAG chunking and vectorization"]
    end
    HTML --> AI[AI bot consumption]
    JSONLD --> AI
    MD --> AI
```

Fig 6-2: Three complementary layers — pure HTML for coarse-grain crawling, JSON-LD for knowledge graphs, Markdown for RAG.

What each layer looks like

(1) Pure HTML skeleton

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Acme Aesthetics - downtown Chicago laser clinic</title>
  <meta name="description" content="Acme Aesthetics offers laser...">
  <link rel="canonical" href="https://acme-aesthetics.example/">
</head>
<body>
  <main>
    <h1>Acme Aesthetics</h1>
    <section>
      <h2>About</h2>
      <p>Founded in 2018, Acme focuses on...</p>
    </section>
    <section>
      <h2>Services</h2>
      <ul>
        <li>Laser hair removal</li>
        <li>Botulinum treatments</li>
      </ul>
    </section>
  </main>
</body>
</html>
```

(2) Schema.org JSON-LD — see Ch 7 for the full design.

(3) Markdown raw-text block — for RAG chunking:

```markdown
# Acme Aesthetics

## About
```

```
Founded in 2018 in downtown Chicago, specializing in...

## Services
- Laser hair removal
- Botulinum treatments
```

All three layers coexist in the same URL response: HTML as the body, JSON-LD in a `<script type="application/ld+json">` block, Markdown in a `<script type="text/markdown" id="axp-markdown">` block.

6.3 Cloudflare Worker injection

AXP is delivered by **edge injection**: the CDN layer intercepts requests and chooses the response body based on User-Agent. Our platform uses Cloudflare Workers.

Fig 6-3: Worker routing decision flow

```
flowchart TD
    Req[HTTP request] --> UA{UA matches<br/>AI bot?}
    UA -->|yes| Cache{Worker cache<br/>hit?}
    UA -->|no| Pass[pass through<br/>to customer origin]
    Cache -->|hit| Serve[return cached shadow doc]
    Cache -->|miss| Fetch[fetch shadow doc from<br/>geo.baiyuan.io API]
    Fetch --> Store[write to Worker cache<br/>TTL 15 min]
    Store --> Serve
    Pass --> Origin[customer origin server]
```

Fig 6-3: Worker tries cache first, only fetches from origin API on miss. Human requests pass through with no added latency.

Worker pseudo-code

```javascript
export default {
  async fetch(request, env) {
    const ua = request.headers.get('user-agent') || '';
    const url = new URL(request.url);

    // 1. Not an AI bot: pass through to customer origin
    if (!isAIBot(ua)) {
      return fetch(request);
    }

    // 2. AI bot: try cache
    const cacheKey = `axp:${url.hostname}:${url.pathname}`;
    const cached = await env.KV.get(cacheKey);
    if (cached) {
      return new Response(cached, {
        headers: { 'content-type': 'text/html; charset=utf-8' },
      });
    }

    // 3. Cache miss: fetch shadow doc from our API
    const axpUrl = `https://api.geo.baiyuan.io/axp?host=${url.hostname}&path=${url.pathname}`;
    const axpRes = await fetch(axpUrl);
```

```
    if (!axpRes.ok) {
      return fetch(request); // shadow fetch failed - fall back to origin
    }

    const body = await axpRes.text();
    await env.KV.put(cacheKey, body, { expirationTtl: 900 });
    return new Response(body, {
      headers: { 'content-type': 'text/html; charset=utf-8' },
    });
  },
};
```

Key design points:

- AI-bot requests and human requests **take entirely different paths**
- If shadow-document fetch fails for any reason, **fall back to the origin** — we never let customer AI traffic 404
- 15-minute TTL is the chosen trade-off between freshness and backend load

---

6.4 AI bot UA list and detection strategy

The platform currently recognizes **25 AI bot UAs**, grouped by function:

Fig 6-4: AI bot UA groups

```
flowchart LR
    subgraph LLM["LLM crawlers (10)"]
      GPTBot
      ClaudeBot
      GoogleExtended[Google-Extended]
      CCBot[CCBot / Common Crawl]
      MetaBot[FacebookBot / Meta-ExternalAgent]
      ByteBot[Bytespider]
      AnthropicBot[anthropic-ai]
      AppleBot[Applebot-Extended]
    end
    subgraph Search["Search-type crawlers (6)"]
      Perplexity[PerplexityBot]
      ChatGPTUser[ChatGPT-User]
      PerplexityUser[Perplexity-User]
      YouBot[YouBot]
      PhindBot
    end
    subgraph Preview["Preview / link generators (5)"]
      LinkedInBot
      FacebookExt[facebookexternalhit]
      TwitterBot[Twitterbot]
      DiscordBot[Discordbot]
    end
    subgraph RAG["RAG / enterprise crawlers (4)"]
      CohereBot[cohere-ai]
      DiffBot[Diffbot]
      OmigoBot[Omigo]
    end
```

Fig 6-4: 25 AI bots grouped by role. The platform enables AXP injection for all four groups by default; admins can disable groups per customer if needed.

Detection strategy

Implementation uses a **combined regex** rather than nested `if/else`, for maintainability:

```
const AI_BOT_REGEX = new RegExp(
  [
    'GPTBot', 'ChatGPT-User', 'OAI-SearchBot',
    'ClaudeBot', 'anthropic-ai', 'Claude-Web',
    'Google-Extended', 'GoogleOther',
    'PerplexityBot', 'Perplexity-User',
    'CCBot', 'Bytespider', 'FacebookBot',
    'Meta-ExternalAgent', 'Applebot-Extended',
    'cohere-ai', 'Diffbot', 'YouBot', 'PhindBot',
    'LinkedInBot', 'facebookexternalhit', 'Twitterbot',
    'Discordbot', 'Omigo', 'DuckAssistBot',
  ].join('|'),
  'i'
);

function isAIBot(ua) {
  return AI_BOT_REGEX.test(ua);
}
```

The UA list is reviewed quarterly; new crawlers (e.g., `OAI-SearchBot` first observed in July 2025) must be added promptly.

6.5 Path conflicts for SaaS self-brands

A practical edge case: **when the SaaS platform itself is also a user of the SaaS (dogfooding)**, the same domain needs to serve both *"platform users"* (logged-in product) and *"brand website visitors"* (anonymous content).

Baiyuan's own `geo.baiyuan.io` is exactly this case:

Path	Human user	AI bot
`/`	Marketing home (public)	AXP brand page for "Baiyuan"
`/dashboard`	Authenticated dashboard (private)	403 — should not be AXP-ified
`/features`, `/pricing`	Product pages (public)	AXP service pages
`/login`, `/signup`	Auth pages (public but no brand content)	Do not inject; pass through

Decision tree

```
flowchart TD
    R[Request] --> B{AI bot?}
    B -->|no| P1[pass through to origin]
    B -->|yes| P{Path category}
    P -->|marketing / brand page| A[inject AXP]
    P -->|authenticated feature| X1[return 403 Forbidden]
    P -->|public page without brand content| X2[pass through to origin]
```

Fig 6-5: The path-category table is maintained per-brand in admin settings. Paths not in the table default to pass-through — conservative by design.

6.6 Automatic sitemap generation

AI bot crawl efficiency depends on `sitemap.xml`. Under AXP, the sitemap must be **dynamically generated in alignment with AXP paths**, otherwise we end up in the confusing state of *"sitemap lists URL X, but the Worker does not inject AXP on path X."*

The platform generates a sitemap per customer domain following these rules:

- URLs derive from `brand_locations`, `brand_services`, `brand_employees` tables
- Each URL gets a `<lastmod>` from the corresponding entity's `updated_at`
- `<priority>` by path type: home 1.0, service pages 0.8, employee pages 0.6
- `robots.txt` actively declares `Sitemap: https://<domain>/sitemap.xml`

The sitemap is also served by the Cloudflare Worker; human visitors that hit `/sitemap.xml` see it too (SEO convention — not hidden).

6.7 JSON-LD flattening pitfalls

The Schema.org spec allows **nested arrays**, but in practice the following pattern triggers issues:

Fig 6-6: wrong vs right, side by side

```
// WRONG: nested array - some AI parsers reject the whole block
{
  "@context": "https://schema.org",
  "@graph": [
    [
      { "@type": "Organization", "name": "Acme Aesthetics" }
    ],
    [
      { "@type": "Service", "name": "Laser treatment" }
    ]
  ]
}

// RIGHT: flat array with @id references
{
  "@context": "https://schema.org",
  "@graph": [
    { "@type": "Organization", "@id": "#org", "name": "Acme Aesthetics" },
    { "@type": "Service", "@id": "#svc-laser", "name": "Laser treatment",
      "provider": { "@id": "#org" } }
  ]
}
```

Fig 6-6: Relationships between entities must be expressed via `@id` references, not via array nesting. This is a hard requirement of the Schema.org validator tooling.

Benefits of flattening + `@id` linking:

- Google's Rich Results tool validates cleanly

- Wikidata and Wikipedia structured-data extractors align to the same shape
- AI knowledge-graph construction becomes more stable

6.8 GSC indexing field notes

A record of issues encountered with Google Search Console (GSC) during 2024–2025 operation:

Pitfall	Symptom	Root cause	Remedy
`noindex` meta override	GSC reports "excluded by noindex tag"	UAT `.env` accidentally deployed to PROD	Strict env-var validation at app startup refusing unreasonable combinations
Cross-domain canonical	PROD page's canonical points to UAT domain	Same codebase uses shared canonical logic	Change canonical to dynamic, from `request.hostname`
Missing bot UA	GSC index shifts but AI citations drop	New bot UA not yet in regex	Quarterly review of CF Worker logs for unmatched UAs
Sitemap inconsistency	GSC reports "Discovered — currently not indexed"	AXP page exists but sitemap missed it	Generate sitemap from the same source (AXP index table)
HTTPS/HTTP mixing	`robots.txt` returns 200 on HTTP, 404 on HTTPS	Worker did not handle `http://` traffic	Force 301 to HTTPS and inject robots on both

These issues are not AXP-specific, but **AXP amplifies their severity**: AI bots recrawl less frequently than Googlebot, so a single mistake can cost weeks before it is observed and corrected. It is better to run a pre-flight `check-prod-seo.sh` in CI than to find these live.

6.9 RAG knowledge base integration: per-brand KB automation

AXP page quality depends not just on "can AI bots fetch it" but on **how much factual brand knowledge is inside**. Pages generated purely from LLM inference tend to contain hallucinations or vague generalities. Injecting knowledge from the brand's own RAG knowledge base yields a qualitatively different result.

6.9.1 Brand isolation: one dedicated KB per brand

Baiyuan uses a central shared RAG engine (§9.4), but each brand's documents live in a **dedicated Knowledge Base (KB)** identified by `rag_kb_id`:

- Multiple brands under the same tenant (e.g. a cosmetics brand and a food brand) never share knowledge
- AXP generation passes `kbId` to the query, ensuring only that brand's facts are retrieved
- When a brand is deleted, its KB can be cleaned up with no residual data risk

```
flowchart LR
    subgraph Tenant["Same tenant"]
      BrandA["Brand A<br/>rag_kb_id: kb-aaa"]
      BrandB["Brand B<br/>rag_kb_id: kb-bbb"]
    end
```

```
subgraph RAG["Central RAG engine"]
  KBA["KB: kb-aaa<br/>(Brand A only)"]
  KBB["KB: kb-bbb<br/>(Brand B only)"]
end
BrandA -->|askWithKbId| KBA
BrandB -->|askWithKbId| KBB
```

Fig 6-7: Brand-level KB isolation. The engine is shared; the knowledge is not.

6.9.2 `seedBrandRAGKB`: automatic KB creation and seeding on AXP enable

When a brand enables AXP (`enableAXP` API), the system runs three steps asynchronously in the background:

1. **Create KB** (if absent) — `ragCreateKnowledgeBase` returns a new `kbId`, written to `brand_rag_configs`
2. **Upload brand profile** (text document) — brand name, industry, description, website, and core keywords are serialised as a structured text document, providing the KB's anchor knowledge
3. **Upload website page URLs** (up to 20, ordered by `geo_importance` DESC) — the RAG backend crawls and vectorises each page, supplementing static profile knowledge with live site content

```
// Inside enableAXP setImmediate block (non-blocking)
await initialCrawl(brandId, tenantId);   // generate AXP pages
await seedBrandRAGKB(brandId, queryFn);  // parallel: seed RAG KB
```

Design principles: - **Non-blocking** — seeding does not delay the immediate AXP page generation response - **Idempotent** — repeated calls only add new documents; no duplicate KB is created - **Graceful degradation** — any URL crawl failure is silently skipped; other URLs continue

6.9.3 Keyword injection: giving AXP content a brand voice

`brand.keywords` (the brand's target GEO/SEO keywords) are injected into every RAG query as a `keywordsHint` suffix:

```
// hybridCoordinator.service.js
const keywordsHint = keywords.length
  ? `\n\nPlease naturally incorporate the following target keywords (not all need to appear): ${keywords
  : '';
const question = PAGE_TYPE_QUESTIONS[pageType](brandName) + keywordsHint;
```

For `pricing_summary` and `product_features` — page types prone to "pure tables, zero keywords" — the RAG prompt explicitly requires a **keyword-rich introductory paragraph** before the table:

```
1. Opening paragraph (2-3 sentences): describe the brand's positioning,
   naturally incorporating the target keywords
2. Full pricing table: only list data confirmed in the knowledge base;
   do not infer or fabricate
```

Observed keyword coverage (`ILIKE` substring match, 5 brands × 6 page types, 2026-04-21):

Brand	Keywords	Lowest coverage page	Min coverage	Most pages
Baiyuan GEO	13	pricing_summary	9/13	11–13/13
Baiyuan RAG KB	12	pricing_summary	7/12	11–12/12
aiLife	10	pricing_summary	7/10	9–10/10
PIF	12	faq / pricing	7/12	10–12/12
Baiyuan Technology	10	pricing_summary	1/10 ☐	9–10/10

☐ Brand website has no public pricing page; RAG correctly refuses to fabricate data — low coverage is expected.

6.9.4 `content_preview`: a per-page quality signal

The AXP page list API now includes a `content_preview` field: the first 150 characters of `content_md` after stripping multiline HTML comments.

This fixes a practical problem: when all six page types of the same brand displayed the same `fingerprint_phrase`, users could not confirm whether each page had generated correctly.

```
-- Strip multiline HTML comments; take first 150 chars
LEFT(REGEXP_REPLACE(content_md, '<!--[\s\S]*?-->', '', 'g'), 150) AS content_preview
```

The regex must use `[\s\S]*?` (dotall) rather than `[^>]*` — the latter fails to match HTML comments that span multiple lines.

Key takeaways

- A single HTML cannot serve both human experience and AI parseability — AXP is the decoupling mechanism
- AXP has three layers: pure HTML skeleton + Schema.org JSON-LD + Markdown raw text
- Cloudflare Worker detects AI bot UA at the edge; AI and human requests follow completely different paths
- 25 AI bot UAs maintained as a single regex; reviewed quarterly for new crawlers
- SaaS self-brand path conflicts require an admin-maintained classification table; default to conservative pass-through
- Sitemap must be dynamically generated in alignment with AXP; JSON-LD uses `@id` flattening, not nested arrays
- GSC issues amplify under AXP — a pre-flight check script in CI is the cheapest preventive control
- Each brand has an isolated RAG KB (`rag_kb_id`); `seedBrandRAGKB` automatically uploads a brand profile and website URLs when AXP is enabled — no manual intervention required
- `brand.keywords` are injected into RAG queries as `keywordsHint`; pricing and feature pages require a keyword-rich intro paragraph; `content_preview` replaces `fingerprint_phrase` as the per-page quality signal

References

- Ch 7 — Schema.org Phase 1: 25 industries × three-layer @id
- Ch 8 — GBP API integration strategy
- Cloudflare. *Workers runtime documentation*. https://developers.cloudflare.com/workers/
- OpenAI. *GPTBot user agent*. https://platform.openai.com/docs/gptbot
- Anthropic. *ClaudeBot documentation*. https://support.anthropic.com/en/articles/8896518
- Google. *Google-Extended and generative AI training*. https://developers.google.com/search/docs/crawling-indexing/overview-google-crawlers

---

Chapter 7 — Schema.org Phase 1: 25 Industries × Three-Layer @id Interlinking

> Schema.org is not "adding a few tags." Without industry specialization, without entity interconnection, without auto-generation, it is effectively invisible to AI.

Table of Contents

- 7.1 Schema.org's role has shifted in the AI era
- 7.2 Industry-specialized @type across 25 categories
- 7.3 Three-layer @id interlinking
- 7.4 Physical vs online: divergent field weights
- 7.5 Data completeness algorithm
- 7.6 Dual entry points: Wizard + Edit
- 7.7 GBP URL parser
- 7.8 Function skeleton
- Key takeaways
- References

7.1 Schema.org's role has shifted in the AI era

Schema.org was created in 2011 by Google, Bing, Yahoo, and Yandex as a shared structured-data vocabulary. Its original purpose was to feed **traditional search engines** into producing Rich Results (star ratings, breadcrumb trails, expandable FAQ, etc.).

Since 2024 its role has shifted in two fundamental ways:

1. **From "search-engine decoration" to "structured source for AI training data"** — major LLMs ingest Common Crawl during pretraining, and Schema.org JSON-LD is the densest entity-data layer in that corpus.
2. **From "nice-to-have" to "required"** — a website without Schema.org looks to AI like *"a blob of text"*; a website with Schema.org looks like *"an identifiable entity"*. The gap is the same order as *"does this image have alt text"* for screen readers.

This book treats Schema.org as **the first lever in Baiyuan GEO's optimization path**. Without a solid Schema.org structure, other dimensions cannot stabilize AI perception no matter how they are tuned.

7.2 Industry-specialized @type across 25 categories

Schema.org defines hundreds of `@type` values, many highly specialized (e.g., `MedicalClinic`, `VeterinaryCare`, `CafeOrCoffeeShop`). **Picking the wrong `@type` is the equivalent of filing yourself under the wrong cabinet** — AI uses `@type` as a key dimension when placing an entity in its knowledge graph.

Our platform distills common industries into **25 categories**, each mapping to a primary + secondary Schema.org `@type`.

Fig 7-1: 25-industry classification (16 physical + 7 online + 2 fallback)

code	Name	Schema.org `@type`
`medical_clinic`	Medical / aesthetics clinic	`MedicalClinic`, `LocalBusiness`

code	Name	Schema.org `@type`
`dental_clinic`	Dental clinic	`Dentist`, `LocalBusiness`
`general_clinic`	General medical clinic	`MedicalOrganization`, `LocalBusiness`
`beauty_salon`	Beauty / hair salon	`BeautySalon`, `LocalBusiness`
`fitness`	Gym / yoga / pilates	`HealthClub`, `SportsActivityLocation`
`restaurant`	Restaurant	`Restaurant`, `FoodEstablishment`
`cafe`	Cafe	`CafeOrCoffeeShop`
`legal_service`	Law firm	`LegalService`, `ProfessionalService`
`accounting`	Accounting firm	`AccountingService`, `ProfessionalService`
`real_estate`	Real estate agency	`RealEstateAgent`, `ProfessionalService`
`auto_repair`	Auto repair	`AutoRepair`, `AutomotiveBusiness`
`education_offline`	Tutoring / training center	`EducationalOrganization`, `LocalBusiness`
`veterinary`	Veterinary clinic	`VeterinaryCare`, `MedicalOrganization`
`lodging`	Hotel / B&B	`LodgingBusiness`, `Hotel`
`retail_store`	Retail store	`Store`, `LocalBusiness`
`financial_service`	Financial service	`FinancialService`, `ProfessionalService`
`saas_application`	SaaS product	`SoftwareApplication`, `Organization`
`web_application`	Web tool	`WebApplication`, `Organization`
`mobile_app`	Mobile app	`MobileApplication`, `Organization`
`ecommerce`	Pure e-commerce	`OnlineStore`, `Organization`
`online_education`	Online learning platform	`EducationalOrganization`
`news_media`	News / content site	`NewsMediaOrganization`
`online_professional`	Online professional service	`ProfessionalService`, `Organization`
`other_physical`	Other physical business	`LocalBusiness`
`other_online`	Other online service	`Organization`

Fig 7-1: 16 physical + 7 online + 2 fallback. Each category specifies two `@types` (primary + secondary) exploiting Schema.org's permission for arrays.

Why 25 categories and not more

Schema.org includes hundreds of subtypes. But **over-specialization actually lowers AI recognition rates**. The reasons:

- AI models weight **common types** more heavily during training (e.g., `Restaurant` is more strongly recognized than `FastFoodRestaurant`)
- Too many choices ☐ customers give up filling the form; 25 is the pragmatic balance between coverage and usability
- Specialized subtypes can be added as *additional type* alongside the primary — not forced on every customer

---

7.3 Three-layer @id interlinking

Fig 7-2: Three-layer entity knowledge graph

```
flowchart TB
    subgraph L1["Layer 1: the subject"]
      Org["Organization / LocalBusiness<br/>@id = #org<br/>name / description / url / logo /<br/>address
    end
    subgraph L2["Layer 2: services"]
      Svc1["Service<br/>@id = #svc-1"]
      Svc2["Service<br/>@id = #svc-2"]
      SvcN["..."]
    end
    subgraph L3["Layer 3: people"]
      Emp1["Person / Physician<br/>@id = #emp-1"]
      Emp2["Person / Attorney<br/>@id = #emp-2"]
    end
    Svc1 -->|provider| Org
    Svc2 -->|provider| Org
    Emp1 -->|worksFor| Org
    Emp2 -->|worksFor| Org
    Svc1 -->|performer| Emp1
    Svc2 -->|performer| Emp2
    Org -.->|sameAs| Wiki[Wikipedia]
    Org -.->|sameAs| WD[Wikidata]
    Org -.->|sameAs| LI[LinkedIn]
    Org -.->|sameAs| GBP[Google Business Profile]
```

Fig 7-2: Three layers reference each other by `@id` to form a local knowledge graph; external authoritative nodes are linked via `sameAs`.

Why three layers rather than one blob

A common mistake is to stuff everything into a single `Organization`:

```
{
  "@type": "Organization",
  "name": "Acme Aesthetics",
  "employees": [
    { "name": "Dr. Smith", "jobTitle": "Director" }
  ],
  "services": [
    "Laser hair removal", "Double-eyelid surgery"
  ]
}
```

The problem: AI cannot treat *"Dr. Smith"* as an independently referenceable entity (Person); *"Laser hair removal"* is a string, not an entity (Service). A question like *"who performs laser hair removal?"* has no structured answer to reach.

The three-layer `@id` pattern creates addressable entities:

```
{
  "@context": "https://schema.org",
  "@graph": [
    {
      "@type": ["MedicalClinic", "LocalBusiness"],
      "@id": "https://acme.example/#org",
```

```
      "name": "Acme Aesthetics",
      "sameAs": [
        "https://www.wikidata.org/wiki/Q...",
        "https://www.linkedin.com/company/..."
      ]
    },
    {
      "@type": "Physician",
      "@id": "https://acme.example/#emp-1",
      "name": "Dr. Smith",
      "jobTitle": "Director",
      "worksFor": { "@id": "https://acme.example/#org" }
    },
    {
      "@type": "Service",
      "@id": "https://acme.example/#svc-laser",
      "name": "Laser hair removal",
      "provider": { "@id": "https://acme.example/#org" },
      "performer": { "@id": "https://acme.example/#emp-1" }
    }
  ]
}
```

When a user asks AI *"who performs laser hair removal at Acme?"*, the AI has a complete entity chain to reason across — not a fuzzy string match.

7.4 Physical vs online: divergent field weights

The `is_physical` flag determines the completeness weight table. The two types of businesses influence AI citation through completely different dimensions.

Fig 7-3: Weight divergence

```
flowchart LR
    subgraph Physical["Physical (is_physical=true)"]
      P1["address 15%"]
      P2["GBP Place ID 15%"]
      P3["opening hours 10%"]
      P4["phone 10%"]
      P5["services 10%"]
      P6["employees 10%"]
      P7["url / name / logo 30%"]
    end
    subgraph Online["Online (is_physical=false)"]
      O1["url 20%"]
      O2["description 15%"]
      O3["logo_url 10%"]
      O4["services / features 20%"]
      O5["sameAs external links 15%"]
      O6["FAQ 10%"]
      O7["other 10%"]
    end
```

Fig 7-3: For physical businesses, address + GBP dominate at 30%. For online services, url + description dominate at 35%. Same algorithm, two weight tables, reflecting real user intent differences.

Rationale

- **Physical businesses**: user queries to AI often contain a locality (*"best dermatology clinics in downtown Chicago"*). AI needs to extract address and opening-hours info from Schema.org; without those the AI answer cannot "land."
- **Online services**: user queries are capability-oriented (*"what is the best CRM?"*). AI needs description, differentiators, and comparable features; address is irrelevant.

The platform UI hides/shows fields dynamically based on `is_physical`: physical customers see Address and Opening Hours cards; online customers do not.

7.5 Data completeness algorithm

Each field carries a **weight** (0–100); filling it in adds its weight. Total completeness is the weighted average.

```
function computeCompletion(brand, industry) {
  const weights = industry.is_physical ? PHYSICAL_WEIGHTS : ONLINE_WEIGHTS;
  let score = 0;
  let maxScore = 0;

  for (const [field, weight] of Object.entries(weights)) {
    maxScore += weight;
    if (isFilledMeaningfully(brand, field)) {
      score += weight;
    }
  }

  return Math.round((score / maxScore) * 100);
}

// Not just "non-empty" - checks meaningful content
function isFilledMeaningfully(brand, field) {
  const value = getField(brand, field);
  if (!value) return false;
  if (typeof value === 'string' && PLACEHOLDER_PATTERNS.test(value)) return false;
  if (Array.isArray(value) && value.length === 0) return false;
  return true;
}
```

Why "non-empty" is not enough

Early implementation only checked for non-empty fields. Customers started filling `url: "https://"`, `description: "company"`, and other placeholder strings to inflate their completeness score. `isFilledMeaningfully` adds three checks:

1. **Placeholder regex** — catches `^(https?:\/\/)?$`, single-character strings, and known stubs
2. **Minimum length** — e.g., descriptions must be at least 20 characters to count
3. **Format validation** — URLs must be resolvable, phones must parse to E.164 format, etc.

The UI does not prevent the entry, but the algorithm does not count the score. This avoids misleading users into false improvement signals on subsequent optimization work.

7.6 Dual entry points: Wizard + Edit

Fig 7-4: Entry-point flow

```
flowchart TD
    Start{User type} -->|new brand| Wiz[Wizard<br/>linear 7-step flow]
    Start -->|existing brand| Dash[Dashboard<br/>completeness banner]
    Wiz --> W1[Step 1: basic info]
    W1 --> W2[Step 2: industry and description]
    W2 --> W3[Step 3: address & location<br/>if is_physical]
    W3 --> W4[Step 4: opening hours<br/>if is_physical]
    W4 --> W5[Step 5: services]
    W5 --> W6[Step 6: employees]
    W6 --> W7[Step 7: FAQ and social]
    W7 --> Done[done]
    Dash -->|<80%| Alert[red / amber warning]
    Dash --> Edit[/brands/:id/entity<br/>jump to any card]
    Alert --> Edit
    Edit --> Save[save → completeness %<br/>updates live]
```

Fig 7-4: New brands go through Wizard to guarantee first-time coverage; existing brands use Edit to update at will. Both paths share the same Card components (DRY).

Why the Wizard does not force every field

Each Wizard step allows *"skip for now"*:

- **Filling fatigue** would cause customers to abandon the entire flow; accepting ~60% now beats chasing 100% later at the cost of zero
- **Guided UI** beats **mandatory UI** for user-friendliness (progressive disclosure principle)
- After the Wizard, the Dashboard banner continues to nudge unfilled fields — the **second chance for completion**

This is a product-philosophy choice: **let the brand exist in AI first, then chase perfection**.

7.7 GBP URL Parser

Google Business Profile (GBP) exposes location identity through three different ID forms, and customers often only have one of the three URLs handy:

ID type	Example URL	Use
`place_id`	`https://www.google.com/maps/place/?q=place_id:ChIJ...`	Places Details API primary key
`FTID`	`https://maps.google.com/maps?ftid=0x0:0xe6...`	Google Maps internal ID
`CID`	`https://www.google.com/maps?cid=...`	Customer ID short URL form

Fig 7-5: Parser decision tree

```
flowchart TD
    In[Paste any GMB URL] --> Split{URL form}
```

```
    Split -->|contains place_id:| P[extract place_id]
    Split -->|contains ftid=| F[extract FTID<br/>convert to place_id]
    Split -->|contains cid=| C[extract CID<br/>Places API → place_id]
    Split -->|short URL goo.gl| R[resolve 301 → re-enter Split]
    Split -->|other| X[return null<br/>ask user for full URL]
    P --> Done[return Place ID]
    F --> Done
    C --> Done
```

Fig 7-5: The parser branches explicitly on the four URL forms. Any unparseable URL returns a clear error — no guessing.

Why CID requires an API call

CID is a Google-internal serial number and cannot be converted to a Place ID without calling Google's Places API (`findPlaceFromText`):

```javascript
async function cidToPlaceId(cid) {
  const res = await fetch(
    `https://maps.googleapis.com/maps/api/place/findplacefromtext/json?` +
    `input=cid:${cid}&inputtype=textquery&fields=place_id&key=${API_KEY}`
  );
  const data = await res.json();
  return data.candidates?.[0]?.place_id ?? null;
}
```

This call consumes Google API quota; the parser caches per-URL results for 24 hours to avoid repeat consumption.

7.8 Function skeleton

generateBrandEntitySchema

```javascript
function generateBrandEntitySchema(brand, industry) {
  const base = `https://${brand.primary_domain}`;
  const graph = [];

  // Layer 1: Organization / LocalBusiness
  graph.push({
    '@type': industry.schema_types, // array, e.g. ["MedicalClinic", "LocalBusiness"]
    '@id': `${base}/#org`,
    name: brand.name,
    url: brand.url,
    description: brand.description,
    logo: brand.logo_url,
    ...(industry.is_physical && {
      address: buildAddress(brand.location),
      telephone: brand.location?.telephone,
      openingHoursSpecification: buildHours(brand.hours),
      geo: buildGeo(brand.location),
    }),
    sameAs: buildSameAs(brand), // Wikipedia / Wikidata / LinkedIn / GBP
  });

  // Layer 2: services
```

```
  for (const svc of brand.services ?? []) {
    graph.push({
      '@type': 'Service',
      '@id': `${base}/#svc-${svc.slug}`,
      name: svc.name,
      description: svc.description,
      provider: { '@id': `${base}/#org` },
    });
  }

  // Layer 3: employees
  for (const emp of brand.employees ?? []) {
    graph.push({
      '@type': emp.specialized_type ?? 'Person', // Physician / Attorney / ...
      '@id': `${base}/#emp-${emp.slug}`,
      name: emp.name,
      jobTitle: emp.job_title,
      worksFor: { '@id': `${base}/#org` },
    });
  }

  return {
    '@context': 'https://schema.org',
    '@graph': graph,
  };
}
```

This function is the shared foundation for AXP generation (Ch 6) and closed-loop hallucination remediation (Ch 9).

Key takeaways

- Schema.org's role has shifted from "search-engine decoration" to "AI training-corpus structured source"
- 25-industry enum (16 physical + 7 online + 2 fallback) balances coverage and usability
- Three-layer `@id` interlinking (Organization / Service / Person) turns blob data into addressable entities
- The `is_physical` flag triggers different weight tables — physical emphasizes address/GBP, online emphasizes url/description
- Completeness algorithm filters placeholders and form-fills to avoid score inflation
- Wizard guides new brands; Edit serves existing brands; shared Card components (DRY)
- GBP URL Parser supports place_id / FTID / CID, with Places API used only when needed

References

- Ch 6 — AXP Shadow Document
- Ch 8 — GBP API Integration
- Ch 9 — Closed-Loop Hallucination Remediation
- Schema.org. *Schema.org vocabulary specification*. https://schema.org/docs/schemas.html
- W3C. *JSON-LD 1.1*. https://www.w3.org/TR/json-ld11/
- Google Maps Platform. *Places API documentation*. https://developers.google.com/maps/documentation/places/web-service

---

Chapter 8 — GBP API Integration Strategy: One-Way Sync from Source of Truth to External Output

> The information about a physical business is scattered — website, Facebook page, LINE account, Google Business Profile, directory profiles. Without a single source of truth, what AI sees is always a collage of contradictions.

Table of Contents

- 8.1 Why GBP is the source of truth for physical businesses
- 8.2 GBP API approval gate and process
- 8.3 Hosting model choice
- 8.4 Field-mapping table
- 8.5 Sync frequency and quota
- 8.6 Remedy for no-webhook
- 8.7 Phase 1–4 roadmap
- Key takeaways
- References

8.1 Why GBP is the source of truth for physical businesses

For physical businesses, Google Business Profile (GBP) is the most authoritative entity-identity node. Three reasons:

1. **Google Search + Maps user surface** — searches like *"best dentist near downtown"* display the GBP location card first; it gets the first click.
2. **AI training data pipeline** — Google AI Overview, Perplexity, ChatGPT, and others heavily consume GBP business data when handling geo-aware queries (directly or via proxy).
3. **Cross-Google-product consistency** — Maps, Search, Assistant, AI Overview all draw from the same GBP record.

Fig 8-1: Data flow — GBP □ Schema.org □ AXP □ AI

```
flowchart LR
    GBP[(GBP<br/>source of truth)] -->|API read| Sync[Baiyuan sync service]
    Sync -->|normalize| DB[(Baiyuan DB<br/>brand_*)]
    DB -->|generate| JSONLD[Schema.org JSON-LD]
    JSONLD --> AXP[AXP shadow doc]
    AXP -->|CF Worker injection| AI[AI bot]
    AXP -->|sitemap.xml| GB[Googlebot]
```

Fig 8-1: Data flows in one direction. GBP edited □ Baiyuan DB follows □ JSON-LD regenerated □ AXP updated □ AI crawls. The customer maintains data in only one place.

Key principle: **the sync is one-way**. Baiyuan platform **does not write back** to GBP (at least in Phase 1–2) to avoid dual-write conflicts and accidental overrides. Phase 3 will open up the LocalPosts write path (see §8.7).

8.2 GBP API approval gate and process

The GBP API is not available by self-registration. It requires an approval process. Noteworthy checkpoints:

Stage	Content	Timeline	Notes
1. Prerequisites	Own a Google Cloud project + at least one verified GBP	1 day	If no existing GBP, use a managed brand's GBP
2. Submit application	Use-case description, estimated QPM, data purpose	Half a day	Use case must focus on "tools for business owners to manage their own profile"
3. Google review	Google team assesses eligibility	7–10 business days	Officially stated; real timelines can run longer
4. Approval	Quota granted, relevant scopes opened	Instant	Default QPM = 300, higher quotas available on further application

Common rejection reasons

- Use case vague (*"for SEO tooling"* is seen as borderline)
- No verified business (applicants must have management rights)
- Submission form entered via public forum rather than the formal channel (this is a confusion trap in Google's own documentation)

Our strategy: **accumulate verified GBPs during Phase 1 (Schema.org manual fill)**, then submit Phase 2's application with 5 verified case studies as eligibility evidence.

8.3 Hosting model choice

To let Baiyuan read a customer's GBP, two hosting models are available:

Fig 8-2: Two hosting models

```
flowchart TB
    subgraph Manager["Model A: Add Manager"]
      C1[Customer adds Baiyuan as Manager<br/>in GBP UI]
      C1 --> S1[Baiyuan uses Service Account<br/>to read the location]
      S1 --> Pro1["+ one-time setup, long-lived<br/>- requires customer to use GBP UI<br/>- Baiyuan sees
    end
    subgraph OAuth["Model B: OAuth"]
      C2[Customer clicks 'Connect Google'<br/>in Baiyuan UI]
      C2 --> S2[OAuth flow obtains<br/>refresh_token]
      S2 --> Pro2["+ familiar UX (like FB connect)<br/>+ fine-grained scope<br/>- refresh_token expires
    end
```

Fig 8-2: Model A is simpler but has a higher user-skill bar. Model B's UX is familiar but has token-maintenance cost.

The platform runs **both models in parallel**: the UI defaults to OAuth (B); if OAuth fails to obtain permissions (e.g., Workspace accounts with management restrictions), it falls back to Manager-add (A).

8.4 Field-mapping table

The GBP data model does not align one-to-one with Schema.org. An **explicit mapping table** is required. Twelve commonly needed mappings:

GBP field	Schema.org property	Conversion rule
`title`	`Organization.name`	Direct map, trim whitespace
`storefrontAddress`	`Organization.address`	Build `PostalAddress` object (split multi-line address)
`primaryPhone`	`Organization.telephone`	Normalize to E.164
`websiteUri`	`Organization.url`	Validate resolvable URL
`regularHours.periods`	`openingHoursSpecification`	Convert day-of-week + open/close times to array
`categories.primaryCategory`	`@type` choice	Map to Baiyuan's 25-industry `industry_code`
`profile.description`	`Organization.description`	Max 750 chars (GBP limit)
`metadata.placeId`	`Organization.identifier` + `sameAs`	`identifier` carries the place_id; `sameAs` carries the Maps URL
`moreHours`	`specialOpeningHoursSpecification`	Split special periods (lunch breaks, holidays)
`attributes`	`amenityFeature` / `hasOfferCatalog`	Assign to different properties by attribute type

GBP field	Schema.org property	Conversion rule
`media.photos`	`image` / `logo`	First photo becomes logo; rest become image array
`reviews`	`aggregateRating` + `review`	Rating + review sample (capped, not all)

Each rule is implemented as a pure function in `gbpToSchema.js` and tested with fixtures that assert expected JSON-LD output.

8.5 Sync frequency and quota

GBP API's default quota is **300 QPM** (queries per minute). The trade-off is between *data freshness* and *quota exhaustion*.

Fig 8-3: Sync frequency matrix

Field type	Sync frequency	Daily QPS	Rationale
Basic info (name, address, hours)	once daily	very low	Low change rate
Opening hours changes	hourly	low	Ad-hoc closures, holiday hours need timely reflection
Photos, attributes	once daily	low	Medium change rate
Reviews	every 10 minutes	medium	Reviews arrive frequently and affect aggregateRating
Q&A	hourly	low	Lower velocity than reviews

Fig 8-3: Each location consumes ~150–200 quota calls per day. A 300 QPM account supports ~2,000 locations concurrently.

When brand count exceeds per-account QPM:

1. **Shard accounts** — apply for Additional QPM Quota (Google will review and raise)
2. **Schedule batches** — move less-urgent syncs to off-peak hours
3. **Prioritize** — low-completeness brands first; high-completeness brands throttled

---

8.6 Remedy for no-webhook

The GBP API **does not provide webhooks**.[4] When a customer edits their GBP, there is no real-time notification path back to Baiyuan. Remedies:

- **Notifications API + Pub/Sub**: Google's "indirect webhook" — location changes trigger a push to a Google Pub/Sub topic; Baiyuan subscribes and can react within minutes.
- **Compare-and-sync**: on each read, compare `metadata.updateTime`; only rebuild JSON-LD if the time has advanced, reducing unnecessary downstream work.
- **Customer-triggered sync**: UI exposes a "sync now" button; customers who just edited GBP can force an immediate pull.

Stacked, these three compress "GBP edit ☐ AI visible" latency to **5–10 minutes**. For most scenarios this is enough. Truly real-time use cases (e.g., *"are they open right now"* queries) would require further work.

8.7 Phase 1–4 roadmap

Fig 8-4: Four phases of GBP integration

```
flowchart LR
    P1["Phase 1<br/> live<br/>Local Schema.org<br/>manual fill + GBP URL paste"]
    P2["Phase 2<br/> waiting on API approval<br/>GBP API read<br/>auto-sync basic info"]
    P3["Phase 3<br/> planned<br/>LocalPosts write<br/>active ClaimReview publication"]
    P4["Phase 4<br/> future<br/>Organization Account<br/>chain-brand batch management"]
    P1 --> P2 --> P3 --> P4
```

Fig 8-4: Four phases progressing from read ☐ write ☐ batch. Phase 1 is live independent of API approval; Phase 2 activates on approval; Phases 3–4 scheduled by business demand.

Milestones by phase

Phase	Features	Dependency	Status
Phase 1	25-industry Schema.org, manual fill, GBP URL ☐ Place ID, Wizard+Edit, AXP injection	None	☐ shipped (v2.19.x)
Phase 2	GBP API read: basic info, opening hours, reviews, photos auto-sync	GBP API approval	☐ under review
Phase 3	LocalPosts write (brand announcements / events), active ClaimReview publication to Google	Phase 2 stable for 3 months	☐ planned
Phase 4	Organization Account: chain brands managed through one entry, batch edit, aggregated metrics	Google enables Organization Account for Partner	☐ future

[4] Google Business Profile API. *Notifications overview*. https://developers.google.com/my-business/content/notification-setup

Phase 1's completion is **not gated** on GBP API approval — the customer pastes a Maps URL, the platform extracts Place ID, and Schema.org produces `sameAs` linking to Google Maps. This lets the platform ship while the Google review runs.

Key takeaways

- GBP is the source of truth for physical businesses; Baiyuan runs one-way sync (GBP ☐ Schema ☐ AXP ☐ AI)
- GBP API approval takes 7–10 business days; applications must frame use case as "tools for business owners"
- Dual-model hosting: OAuth as default, Manager-add as fallback
- 12 explicit field mappings, implemented as pure functions for testability
- Sync frequency tiered by field volatility; 300 QPM supports ~2,000 locations
- No webhook; Notifications API + Pub/Sub compresses latency to 5–10 minutes
- Four-phase roadmap: Phase 1 live, Phase 2 pending approval, Phase 3–4 as demand dictates

References

- Ch 6 — AXP Shadow Document
- Ch 7 — Schema.org Phase 1
- Ch 9 — Closed-Loop Remediation (ClaimReview publish path)

Chapter 9 — Closed-Loop Hallucination Detection and Auto-Remediation

> Detection alone is not enough. Without the *"remediate □ verify □ converge"* loop, hallucinations grow back like weeds.

Table of Contents

- 9.1 Why detection alone is insufficient
- 9.2 Five types of AI hallucination
- 9.3 Primary detection: NLI classification + ChainPoll
- 9.4 Central shared RAG: SaaS infrastructure
- 9.5 L1 LLM Wiki: an active semantic layer
- 9.6 Remediation: ClaimReview generation and multi-path injection
- 9.7 Two-tier rescan loop
- 9.8 Convergence timing and acceptance
- Key takeaways
- References

9.1 Why detection alone is insufficient

Traditional brand-monitoring tools follow the loop: **find problem □ notify customer □ customer figures it out**. This worked in the SEO era because most problems are *"low visibility"* and the customer can improve by writing content or placing links.

AI hallucinations are different:

- Customers **do not know** how to correct an AI's incorrect belief about them
- Even if a customer writes *"correct"* content, AI will not necessarily re-crawl and re-train on it
- Each AI platform has a different data pipeline; fixing one does not fix the others

Conclusion: **handing the problem back to the customer is an abdication**. The platform must provide end-to-end automation from detection to convergence.

Fig 9-1: Six-stage closed loop

```
flowchart LR
    A[1. Detect<br/>extract claims] --> B[2. Compare<br/>against ground truth]
    B --> C[3. Decide<br/>is this a hallucination?]
    C -->|yes| D[4. Generate<br/>ClaimReview]
    D --> E[5. Inject<br/>AXP + RAG]
    E --> F[6. Rescan verify<br/>sentinel + full]
    F -->|converged| Done[close]
    F -.->|still present| D
    C -->|no| Skip[no action]
```

Fig 9-1: The six stages from detection to closure. Any stage's failure does not break the others — the system has local fault tolerance.

9.2 Five types of AI hallucination

Our platform classifies brand-related AI errors into five categories, each with its own detection and remediation strategy.

Fig 9-2: Hallucination taxonomy

```
flowchart TD
    Hallu[AI hallucination]
    Hallu --> E1[1. Entity misidentification]
    Hallu --> E2[2. Business fact error]
    Hallu --> E3[3. Industry miscategory]
    Hallu --> E4[4. Temporal / spatial error]
    Hallu --> E5[5. Attribute error]
    E1 -.->|example| EX1["Taiwan brand described as Japanese company"]
    E2 -.->|example| EX2["product line described as competitor's"]
    E3 -.->|example| EX3["dental clinic categorized as aesthetics"]
    E4 -.->|example| EX4["2025 opening described as '10-year-old business'"]
    E5 -.->|example| EX5["B2B SaaS described as consumer app"]
```

Fig 9-2: The five types are not mutually exclusive; a single AI response may contain more than one. Remediation priorities are assigned by impact.

Remediation strategy by type

Type	Priority	Primary remediation
Entity misidentification	P0	Strengthen Schema.org `sameAs` + ClaimReview + Wikidata link
Business fact error	P0	State correct product line in AXP + ClaimReview
Industry miscategory	P1	Correct `industry_code` + Schema.org `@type` + RAG sync
Temporal / spatial error	P1	Explicit `foundingDate` / `address` + ClaimReview
Attribute error	P2	Strengthen description, add FAQ, correct `audience`

P0 errors cause the brand to be misrecognized and must be fixed fastest. P2 errors are semantic nuances that can accumulate before batched treatment.

9.3 Primary detection: NLI classification + ChainPoll

A naive *"extract claim □ compare to Ground Truth"* lookup runs into two fundamental problems:

1. **GT coverage is low** — no one can manually enumerate every brand fact as a key-value; gaps become false positives
2. **Exact match is too strict** — one fact has many valid phrasings (*"founded 2018"* vs *"established 2018"* vs *"7 years in business"*); string comparison misclassifies correct statements as hallucinations

Our platform's primary detection mechanism is **NLI (Natural Language Inference) three-way classification**. GT comparison is just one of three fallback knowledge sources fed into NLI.

Three-tier knowledge sources (combined into NLI input)

```
flowchart TB
    Resp[AI response text] --> Ext[atomic claim extraction<br/>via LLM]
    Ext --> NLI[NLI classifier]
    subgraph KS["Knowledge sources (three tiers by authority)"]
      K1["<b>L1 brand website</b><br/>live scrape / DB cache 24h<br/>first-party, most authoritative"]
```

```
    K2["<b>L2 RAG knowledge base</b><br/>customer uploads + official docs<br/>semantic retrieval answe
    K3["<b>L3 Ground Truth</b><br/>manual key-value<br/>precise but low coverage"]
  end
  K1 --> NLI
  K2 --> NLI
  K3 --> NLI
  NLI --> Cls[classification<br/>entailment / contradiction /<br/>neutral / opinion]
```

Fig 9-3: Knowledge sources are stacked by authority. All three are combined into a single context fed to NLI. If the combined context is less than 500 characters, we skip detection for this scan (too little evidence to judge).

NLI four-way classification

Class	Meaning	Action
`entailment`	Sources **support** the claim	Fact correct, pass
`contradiction`	Sources **contradict** the claim	**Mark as hallucination**, enter remediation flow
`neutral`	Sources **neither confirm nor refute**	Do not classify (important: neutral is NOT a hallucination)
`opinion`	Subjective value judgment (*"the best"*, *"recommended"*)	Skip, opinion is not a fact

The NLI model also emits a `confidence` (0.0–1.0) and a `severity` (critical / major / minor / info) for each claim.

Why "neutral ≠ hallucination" is the core principle

The single most common design error is *"if sources don't mention it, treat it as false."* But a brand website cannot enumerate every fact; if AI claims *"the company has 50 employees"* and the website has no employee-count page, **that does not mean the number is wrong — only that we cannot verify it**. Treating `neutral` as `contradiction` creates a flood of false hallucinations, triggers unnecessary remediation, and poisons the entire closed loop. In the worst case, AI reads the "corrected" (fabricated) content on the next crawl and learns the wrong thing.

ChainPoll: second-opinion voting for uncertain cases

For claims with `confidence` `[0.5, 0.8]` (the uncertain zone), we activate **ChainPoll majority voting**:

```
// Call LLM three times with the same prompt for the same claim;
// take majority result.
async function chainpollVerify(claim, knowledgeContext) {
  const votes = { contradiction: 0, entailment: 0, neutral: 0 };
  const prompt = buildNLIPrompt(claim, knowledgeContext);

  const results = await Promise.allSettled([
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
  ]);

  for (const r of results) {
    if (r.status !== 'fulfilled') continue;
```

```
    const text = (r.value.text || '').toLowerCase();
    if (text.includes('contradiction')) votes.contradiction++;
    else if (text.includes('entailment')) votes.entailment++;
    else votes.neutral++;
  }

  return pickMajority(votes); // trust only 2-of-3 or better
}
```

ChainPoll reduces single-LLM classification noise (the classifier itself is an LLM with randomness). High-confidence (> 0.8) and low-confidence (< 0.5) claims do not trigger ChainPoll — only the ambiguous band. Cost is therefore bounded.

Severity tiers

Once a claim is classified `contradiction`, its `severity` determines the remediation priority:

Severity	Criteria	Remediation schedule
`critical`	Company name, product category, country/location entirely wrong	Immediate remediation, injected within 24h
`major`	Core features, pricing wrong	Remediate within 24h
`minor`	Secondary feature, spec deviation	Batched on next scan
`info`	Phrasing imprecise	Accumulated and treated in batch

This tiering lets resources focus on hallucinations that cause **real business damage**, rather than being drowned in minor wording issues.

9.4 Central shared RAG: SaaS infrastructure

The **L2 RAG knowledge base** from §9.3 is not deployed per-tenant. The entire Baiyuan SaaS platform **shares a single central RAG engine** (hereafter *"Central RAG"*). This is a quietly critical architectural decision.

Why shared central RAG rather than per-tenant RAG

Dimension	Per-tenant RAG	Central shared (our choice)
Operational cost	Each tenant needs independent deploy / scale	Single cluster to maintain
Inference cost	Each tenant runs its own embedding / LLM	Shared GPU / API, amortized
Knowledge-graph coordination	Tenants cannot cross-verify	Central can cross-reference (de-identified)
Upgrade velocity	Per-tenant upgrades	One upgrade, whole platform synced
Isolation risk	Natural isolation	Must engineer tenant isolation (X-Tenant-ID + data ACL)

We chose central shared — trading engineering work for operational efficiency. Tenant isolation uses three layered mechanisms:

1. **HTTP header** `X-Tenant-ID` — every query carries the tenant identity; the RAG engine filters accessible documents accordingly
2. **API key signature** — every SaaS backend call carries `X-RAG-API-Key`; the RAG side validates the caller
3. **RLS / document ACL** — every ingested document is labeled with its owner tenant; queries enforce filtering

These three are analogous to the RLS + app-level double insurance from §2.5. Any single-layer slip still requires the other two to simultaneously fail before cross-tenant leakage happens.

> **Brand-level isolation (updated 2026-04-21)**: within a tenant, multiple brands are further separated by a per-brand Knowledge Base ID (`rag_kb_id`). AXP generation passes the brand's `kbId` to the query, ensuring that different brands under the same tenant (e.g. a cosmetics brand and a food brand) never share factual knowledge. When a brand enables AXP, `seedBrandRAGKB` automatically creates the KB and uploads a brand profile + website page URLs — see §6.9.

Fig 9-4: Central shared RAG architecture

```
flowchart LR
    subgraph Platform["Baiyuan GEO SaaS (application layer)"]
      T1[Tenant A backend]
      T2[Tenant B backend]
      T3[Tenant N backend]
    end
    subgraph RAG["Central RAG engine"]
      direction TB
      Gate["API Gateway<br/>X-Tenant-ID + X-RAG-API-Key validation"]
      subgraph Core["Engine core"]
        W["<b>L1 LLM Wiki</b><br/>semantic processing layer"]
        V["<b>L2 RAG</b><br/>vector retrieval"]
        W --> V
      end
      Gate --> Core
    end
    T1 -->|POST /api/v1/ask<br/>X-Tenant-ID: A| Gate
    T2 -->|POST /api/v1/ask<br/>X-Tenant-ID: B| Gate
    T3 -->|POST /api/v1/ask<br/>X-Tenant-ID: N| Gate
    V --> Ans[tenant-scoped answer]
```

Fig 9-4: All tenants share the same RAG, but every query is filtered at the Gateway by Tenant ID.

9.5 L1 LLM Wiki: an active semantic layer

The Wiki in this architecture is not a conventional *"document store."* It is a layer **actively maintained by LLMs as a semantic knowledge surface**. It is the infrastructure that makes hallucination detection actually work, so it deserves its own section.

9.5.1 Why not just run vector retrieval over raw documents

The intuitive design: customer uploads their website, FAQs, product pages; we embed and store them; on query we retrieve. This pure-passive RAG approach has three fatal flaws:

- **Document contradictions cannot be resolved** — product page claims *"serving 1M customers"*, FAQ says *"serving 500k brands"*; vector retrieval returns both and lets downstream NLI see two conflicting *"sources"*
- **Temporal dimension is lost** — old and new versions of the same content coexist in the vector DB; there is no mechanism to determine *"which is the current fact"*
- **Summarization is absent** — vector retrieval only returns *"the most similar passages"*, but a brand's complete facts about a topic might be scattered across a dozen passages that retrieval will not integrate

These flaws are tolerable in general RAG applications (like customer-support chatbots). They are fatal in **fact-checking** scenarios.

9.5.2 The LLM Wiki processing flow

LLM Wiki is Baiyuan's home-grown semantic-processing layer, which executes the following on every ingested document:

Fig 9-5: Document lifecycle in LLM Wiki

```
flowchart TB
    In[new document ingest<br/>customer upload / site scrape / API push] --> Norm[normalize<br/>strip fo
    Norm --> Segment[semantic chunking<br/>split by topic]
    Segment --> Ext[entity extraction<br/>subject / predicate / object triples]
    Ext --> Compare{compare against existing Wiki}
    Compare -->|new fact| Add[add to Wiki]
    Compare -->|contradiction| Conflict[flag conflict<br/>keep newest + note old]
    Compare -->|supplement| Merge[merge into more complete description]
    Add --> Reindex[incremental compile<br/>update vector index + KG nodes]
    Conflict --> Reindex
    Merge --> Reindex
    Reindex --> Ready[available for L2 RAG retrieval]
```

Fig 9-5: Wiki does not just store documents; it actively maintains the set of facts "currently to be regarded as true." Every new document triggers a re-evaluation of existing knowledge.

9.5.3 Incremental compilation

Traditional RAG systems often require *"rebuild the entire vector index"* when documents are added or modified, which is too expensive to run frequently. LLM Wiki uses **incremental compilation**:

- **Change-detection granularity**: at the *fact triple* level (e.g., `<Baiyuan, founded, 2024>`), not the document level
- **Recompute only affected regions**: when a new fact enters, only the relevant topic's vectors and knowledge-graph nodes are updated
- **Versioning**: every fact carries `valid_from` / `valid_to` timestamps, enabling time-travel queries
- **Revert capability**: if an ingested document introduces error, the specific fact can be reverted to a previous version

For hallucination detection this means: **within minutes** of injecting a ClaimReview remediation into Wiki, NLI queries start seeing the corrected fact. No waiting for nightly rebuilds or downtime.

9.5.4 Core capabilities of LLM Wiki

The layer provides the following native capabilities in support of fact checking:

- **Context comprehension** — LLMs read whole documents (not just single passages), answering cross-passage / cross-document synthesis questions
- **Citation traceability** — answers carry source-passage IDs for downstream verification

- **Multi-expression aggregation** — different phrasings of the same fact (*"founded 2024"* vs *"established 2024"*) are treated as the same node
- **Automatic summarization** — long documents auto-summarized, reducing downstream retrieval load
- **Conflict management** — new and old sources disagreeing retain the newest and tag the history, rather than muddling together

Baiyuan exposes these capabilities through a thin REST API; the business layer is decoupled from underlying implementation. Any LLM or vector-engine replacement can happen within this layer without affecting business code.

9.5.5 L1 Wiki's enabling role for L2 RAG

L2 RAG vector retrieval appears to be *"standard RAG,"* but its **index content** comes from L1 Wiki's processed facts, not raw documents. The implications:

- Retrieval never returns *"raw passages not processed by Wiki"*
- Results carry a *fact ID* traceable to specific Wiki nodes
- For the NLI classifier, source quality is far higher than it would be reading raw documents

In short: **L1 Wiki is the knowledge ontology; L2 RAG is knowledge retrieval.** Without L1, L2 is just a faster document search tool.

9.6 Remediation: ClaimReview generation and multi-path injection

ClaimReview Schema.org example

```
{
  "@context": "https://schema.org",
  "@type": "ClaimReview",
  "datePublished": "2026-04-18",
  "url": "https://baiyuan.io/claims/founding-year",
  "claimReviewed": "Baiyuan Technology was founded in 2018",
  "itemReviewed": {
    "@type": "Claim",
    "appearance": "AI-generated response",
    "firstAppearance": "2026-04-16"
  },
  "reviewRating": {
    "@type": "Rating",
    "ratingValue": "1",
    "bestRating": "5",
    "alternateName": "False"
  },
  "author": {
    "@type": "Organization",
    "name": "Baiyuan Technology",
    "url": "https://baiyuan.io"
  }
}
```

ClaimReview is a Schema.org standard property — Google, Facebook, Twitter all parse it. It is not Baiyuan's invention; it is an interoperable component of the global fact-checking ecosystem.[5][6]

[5] Schema.org. *ClaimReview*. https://schema.org/ClaimReview
[6] Google Search Central. *Fact check article markup*. https://developers.google.com/search/docs/appearance/structured-

Three injection paths

```
flowchart LR
    CR[ClaimReview] --> P1[Path A<br/>AXP shadow doc]
    CR --> P2[Path B<br/>RAG knowledge base]
    CR --> P3[Path C<br/>GBP LocalPosts *Phase 3]
    P1 --> AI1[crawled by AI bots]
    P2 --> AI2[retrieved by RAG]
    P3 --> AI3[referenced by Google AI Overview]
    style P3 stroke-dasharray: 5 5
```

Fig 9-6: The same ClaimReview enters three channels simultaneously, maximizing the probability that the AI ecosystem re-learns the correct fact. Path C is gated on GBP Phase 3.

9.7 Two-tier rescan loop

After injection, how do we verify the AI *actually updated its belief*? We rescan — but the **frequency must be neither too low** (the customer loses trust) **nor too high** (cost explodes). The platform splits scanning into **two tiers**.

Fig 9-7: Two-tier scan division of labor

```
flowchart TB
    subgraph L1["Tier 1: sentinel scan (4h cycle)"]
      T1[target: search-type AI<br/>Perplexity / ChatGPT Search /<br/>Google AI Overview]
      T2[lightweight: sample 5 indicator queries]
      T3[purpose: quickly confirm<br/>remediation was picked up]
    end
    subgraph L2["Tier 2: full scan (24h cycle)"]
      F1[target: knowledge-type AI<br/>Claude / Gemini / DeepSeek /<br/>Kimi / GPT-4o / etc.]
      F2[deep: 20-60 intent queries<br/>+ 7-dim scoring + fingerprint compare]
      F3[purpose: confirm corpus integration,<br/>convergence decision]
    end
```

Fig 9-7: Tier 1 short-cycle, targeted at AI that crawls live. *Tier 2 long-cycle, targeted at AI with* long retraining windows. *Together they form complete acceptance.*

Why search-type and knowledge-type split differently

- **Search-type AI** (Perplexity, ChatGPT Search, AI Overview) crawls live at query time. Remediations injected into AXP can be picked up within minutes; **a 4-hour sentinel** is enough for verification.
- **Knowledge-type AI** (standard Claude / Gemini / DeepSeek) depends on pretraining corpora or periodically retrained RAG. Remediations need **days** to integrate; **a 24-hour full scan** is appropriate.

Running a uniform cadence across both would produce: search-type verification latency too high (could know in 4h, waiting 24h is wasteful), knowledge-type false negatives (4h hasn't crawled yet, wrongly judged as remediation failure). The split is not an optimization — it is a consequence of two fundamentally different platforms.

Tier 1 and Tier 2 share data storage

Both tiers write results into the same `scan_results` table with a `scan_layer` column to distinguish. Downstream analysis (dashboard trends, competitor comparison) **merges or separates** as needed:

---

data/factcheck

- **Merge** scenario: user looks at *"today's citation rate"* — both tiers represent signals from the same time window
- **Separate** scenario: analyzing *"remediation convergence speed"* — Tier 1 and Tier 2 converge at different timescales; compute separately

---

9.8 Convergence timing and acceptance

Fig 9-8: Typical remediation convergence curve (illustrative)

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Hallucination prevalence after remediation (illustrative)"
    x-axis ["Day 0", "Day 1", "Day 3", "Day 7", "Day 14", "Day 21", "Day 30"]
    y-axis "Hallucination Rate (%)" 0 --> 100
    bar [80, 75, 60, 40, 22, 10, 3]
```

Fig 9-8: Search-type AI typically converges in 1–3 days; knowledge-type AI in 2–4 weeks. This chart is an aggregate illustration; per-case timelines vary considerably.

Convergence decision rule

A hallucination is declared *"resolved"* only when all of the following hold:

1. **N consecutive scans** (N between 3 and 5 depending on strictness) show the claim absent across all target platforms
2. **Equivalent paraphrases** (synonyms, translations, abbreviations) are also absent
3. **Both Tier 1 and Tier 2 have validated** — cannot be a single-tier pass

When all are satisfied, the UI marks the hallucination *"☐ resolved"* with the time to convergence recorded.

Handling non-convergence

If a hallucination persists beyond expected convergence time, the system escalates it to a **stubborn hallucination**:

- Analyze which platforms missed (does this platform depend on a specific corpus source?)
- Check whether the ClaimReview has technical errors (dead URL, malformed JSON-LD)
- Consider additional remediation channels — Wikidata edits, Wikipedia fact tags, LinkedIn company description updates

Stubborn hallucination handling typically requires **human intervention**. This is the current boundary of the automated loop. Stating that boundary honestly is more responsible than pretending full automation exists.

Key takeaways

- Closed-Loop automates the six stages: detect, compare, decide, generate, inject, verify
- AI hallucinations split into five types (entity / fact / industry / temporal / attribute) with priorities
- **Primary mechanism is NLI + ChainPoll**, not simple GT matching; GT is one of three knowledge-source tiers
- *"neutral ≠ hallucination"* is the core design principle; false positives here poison the remediation cycle
- Central shared RAG SaaS architecture: all tenants share one engine with three-layer tenant isolation
- **L1 LLM Wiki** is an active semantic layer — conflict management, incremental compilation, versioning, summarization

- L1 Wiki is the *knowledge ontology*; L2 RAG is *knowledge retrieval*. Without L1, L2 is just fast document search
- ClaimReview injected across AXP + RAG + GBP LocalPosts (3 paths); Wiki incremental compile makes remediation live in minutes
- Two-tier scan (Tier 1 sentinel 4h + Tier 2 full 24h) respects the different cadences of search-type and knowledge-type AI
- Stubborn hallucinations still require human intervention — the current automation boundary

References

- Ch 6 — AXP Shadow Document
- Ch 7 — Schema.org Phase 1
- Ch 10 — Phase Baseline Testing

Chapter 10 — Phase Baseline Testing: Longitudinal AI-Perception Comparison with Fixed Question Sets

> If you ask different questions every time, you can never tell whether the score moved because the brand changed or because the questions changed.

Table of Contents

- 10.1 Regular scan's longitudinal blind spot
- 10.2 Phase baseline testing design
- 10.3 Data-path isolation from regular scanning
- 10.4 Four observation axes
- 10.5 Operational guidance
- Key takeaways
- References

10.1 Regular scan's longitudinal blind spot

The regular scan from Ch 2 **dynamically generates intent queries each run**, to simulate the variety of real user questions. This is good for *horizontal* answers to *"how often is the brand mentioned right now?"* but it **cannot answer the longitudinal question**: *"between last week and this week, how has the AI's perception of this brand changed?"*

Because the two scans used different query sets, the score delta has at least three possible causes:

1. **Real change** — AI's perception of the brand shifted (better or worse)
2. **Query shift** — the new query set happens to trigger more / fewer mentions than the old one
3. **Randomness** — the same query on the same AI can produce slightly different responses across runs

Without separating these three causes, the trend line is **noise**. To filter out real change, we need **a fixed query set with repeated testing**.

Fig 10-1: Regular scan vs baseline testing

```
flowchart TB
    subgraph Regular["Regular scan (horizontal)"]
      R1[new query each run]
      R2[answers 'what's the current score']
      R3[not suited for time-series comparison]
    end
    subgraph Baseline["Phase baseline (longitudinal)"]
      B1[fixed query set]
      B2[repeated weekly / bi-weekly]
      B3[answers 'how is perception evolving']
    end
    Regular --- Baseline
```

Fig 10-1: The two scan types answer different types of questions. Complementary, not substitutable.

10.2 Phase baseline testing design

Phase definitions and cadence

- **Phase 1** (Day 0): establish baseline
  - System generates **20 representative intent queries** based on the brand's industry
  - All questions and full AI responses (`response_text`) stored in `baseline_test_runs.queries_json` and `baseline_test_responses`
  - These 20 queries **are preserved permanently**, immune to any cleanup
- **Phase 2** (Day 7 ±1): first retest
  - Reuse Phase 1's 20 queries; re-ask every platform
  - Every response stored as comparison data
  - Compute score delta for "same question, different time"
- **Phase 3** (Day 14 ±1): second retest
  - Same 20 queries, third asking
  - Three data points give a short-term trend line
- **Further phases** (optional): extend to Phase 4, 5 (monthly) by customer request

Fig 10-2: Three-phase data structure

```
flowchart LR
    Q["Intent queries × 20 (fixed)"] --> P1[Phase 1<br/>Day 0]
    Q --> P2[Phase 2<br/>Day 7]
    Q --> P3[Phase 3<br/>Day 14]
    P1 -->|compare| P2
    P2 -->|compare| P3
    P1 -->|compare| P3
    P1 --> R[response_text × platforms]
    P2 --> R
    P3 --> R
```

Fig 10-2: Three askings, one query set, three complete response sets compared.

Why 20 questions and not more

- **Statistical angle**: 20 questions already cover the main intent types (best-of, comparison, recommendation, situational); more has diminishing returns
- **Cost angle**: 20 × 15 platforms × 3 phases = 900 API calls per brand; more would stress quotas
- **Visualization angle**: three parallel trend lines across >20 questions become hard to read

20 is an empirical choice. It can be revised if data supports a different number later.

10.3 Data-path isolation from regular scanning

Phase baseline testing runs on a **fully independent data path** that does not overlap with regular scanning:

Facet	Regular scan	Phase baseline
Query source	Dynamically generated each run	Fixed at Phase 1
Trigger frequency	Daily / 4h	Scheduled or manually triggered
Included in main GEO score?	Yes	**No** (shown separately)
Subject to Stale Carry-Forward?	Yes	**No** (failed scans mark `incomplete`)
Data retention	Rolling window	**Permanent `response_text` retention**

Facet	Regular scan	Phase baseline
Uses Redis cache?	Yes (reduces duplicate API cost)	**No** (every ask is fresh)

Why baseline bypasses cache

Regular scans cache a recent response for the same question (assuming AI's opinion doesn't shift in minutes) to reduce cost. But baseline's *purpose* is to measure change in AI's opinion — caching would destroy the measurement.

Why baseline is excluded from main score

If baseline results counted toward the main score, Phase 2 and Phase 3 retests would create a **triple-count effect** (same brand counted multiple times in adjacent time windows), polluting the dashboard trend. Keeping them separate preserves score purity.

10.4 Four observation axes

Phase baseline data's value is not only *"score change"* — it supports four independent observation axes.

Fig 10-3: Four-axis change observation matrix

```
flowchart TB
    subgraph Quant["1. Quantitative"]
      Q1[citation rate up/down]
      Q2[position forward/backward]
      Q3[platform coverage widens/narrows]
    end
    subgraph Qual["2. Qualitative"]
      L1[description wording change]
      L2[depth of language]
      L3[added or removed points]
    end
    subgraph Comp["3. Competitive"]
      C1[co-occurring competitor list]
      C2[competitor positional relationships]
      C3[new competitors appear]
    end
    subgraph Sent["4. Sentiment"]
      S1[sentiment distribution shift]
      S2[rise/fall of strong language]
      S3[neutral → positive / negative shift]
    end
```

Fig 10-3: Four orthogonal axes. Quantitative is computable; Qualitative needs qualitative analysis; Competitive needs graph diff; Sentiment needs scoring.

Analysis approach per axis

Quantitative — compute differences directly: score delta, percentage change, trend slope.

Qualitative — diff Phase 1 and Phase 2 `response_text`; highlight "added paragraphs", "removed paragraphs", "replaced phrases". Surface the diff visually to the customer.

Competitive — extract all brand entities in each response; compare sets across phases (new arrivals / departures / retained). Treat as a set-difference over time.

Sentiment — run sentiment classification per sentence; compare distributions. E.g., Phase 1 = neutral 80% / positive 15% / negative 5%, Phase 2 = neutral 60% / positive 30% / negative 10% □ clear sentiment polarization.

10.5 Operational guidance

When to activate Phase baseline

Three recommended triggers:

1. **Customer subscribes to high-tier plan** — automatically create Phase 1; the customer sees longitudinal evolution in the dashboard
2. **Customer ships a major content revision** — manually trigger Phase 1 right after; measure the revision's effect
3. **After Closed-Loop remediation** (see Ch 9) — initiate Phase testing to verify the hallucination actually converged

Baseline invalidation and rebuild

Rebuild (do not extend) a Phase baseline when any of these occur:

- Brand has a **major business change** (merger, spin-off, pivot) — old queries no longer apply
- AI platform releases a **major version** (GPT-5, Claude 4) — scores across versions cannot be compared directly
- Baseline has been running **more than 6 months** without rebuild — question drift accumulates

On rebuild, create a new `baseline_cohort_id`; keep the old for historical reference but do not add new data points to it.

UI presentation principles

- **Do not mix with the regular dashboard** — prevent users from accidentally adding Phase scores to daily scores
- **Dedicated `/baseline` page** showing Phase 1□2□3 comparison views
- **Primary view**: side-by-side response for the same question across three phases (left-center-right columns), diffs highlighted in color
- **Secondary view**: aggregated trend across the four observation axes

---

Key takeaways

- Regular scans generate dynamic queries (good for horizontal "now"); baselines fix queries (good for longitudinal "evolution")
- Phase 1□2□3 re-asks 20 fixed questions three times; `response_text` is retained permanently
- Baseline uses an independent data path — excluded from main score, bypasses cache, not subject to Stale Carry-Forward
- Four observation axes: quantitative (scores), qualitative (text), competitive (entity set), sentiment (distribution)
- Activation triggers: high-tier plan / major content revision / post-Closed-Loop verification
- Baseline rebuild triggers: business change / AI major version / 6+ months old

References

- Ch 2 — System Overview
- Ch 3 — Seven-Dimension Scoring Algorithm
- Ch 4 — Stale Carry-Forward
- Ch 9 — Closed-Loop Hallucination Remediation

Chapter 11 — Five-Brand Field Observations: Six Weeks of Anonymized Data

> Theory is not enough. Data validates. What follows are aggregated observations from operating five live pilot brands for ~6 weeks, customer names and identifiable numbers de-identified.

Table of Contents

- 11.1 Brand portraits (anonymized)
- 11.2 GEO score distribution
- 11.3 Platform coverage asymmetry
- 11.4 Schema.org completeness and citation rate
- 11.5 AXP deployment before/after
- 11.6 Customer-side pitfalls
- 11.7 Three unexpected findings
- 11.8 First-month commercial validation
- Key takeaways
- References

11.1 Brand portraits (anonymized)

The five pilot brands span B2B, B2C, physical, and pure online configurations:

Code	Industry	Type	Market language	Entry GEO score
Brand A	B2B SaaS (marketing tech)	Online	bilingual zh / en	mid-tier
Brand B	Professional financial services	Online	primarily English	high-tier
Brand C	B2B SaaS (knowledge management)	Online	bilingual zh / en	mid-tier
Brand D	Restaurant chain, physical	Physical	Chinese	low-tier
Brand E	Baiyuan Technology itself (dogfooding)	Online	bilingual zh / en	low-tier (cold start)

Entry GEO scores shown as low/mid/high tiers to preserve relative structure while redacting absolute values.

Why these five as observation sample

- **Brand A / C** form a *"comparison pair"*: both B2B SaaS, bilingual, similar entry conditions; contrast downstream optimization
- **Brand B** is the high-tier starting point — does a brand that AI already knows well still have room to improve?
- **Brand D** is the only physical business in the sample — tests the GBP + Schema.org LocalBusiness combination
- **Brand E** is a cold-start extreme: brand new, no external references at all

With only 5 samples we **cannot make statistical claims**. This chapter presents *observations*, not *conclusions* — the aim is to convey the real shape of operation.

11.2 GEO score distribution

Across six weeks all brands saw movement on all seven dimensions.

Fig 11-1: Seven-dimension radar (Week 1 vs Week 6, anonymized aggregate)

```
%%{init: {'theme':'base'}}%%
graph TD
    W1["Week 1 aggregate average<br/>Citation: mid-low<br/>Position: mid<br/>Coverage: low<br/>Breadth:
    W6["Week 6 aggregate average<br/>Citation: mid<br/>Position: mid-high<br/>Coverage: mid<br/>Breadth:
    W1 -.->|6 weeks operation| W6
```

Fig 11-1: Every dimension improved. Coverage and Breadth improved most. Data shown as "low/mid/high" tiers, concrete numbers omitted.

Three observed patterns

1. **Position moves first, Citation follows** — after Schema.org optimization, the *position* at which the AI mentions the brand shifts earlier in the response (from last-paragraph to top-third); only *weeks later* does the mention-count itself rise
2. **Coverage expands faster than Breadth** — adding coverage of intent-query types (comparison, recommendation) is easier than expanding to additional AI platforms
3. **Consistency converges last** — for a brand to look the same on ChatGPT and DeepSeek typically requires 4–8 weeks

---

11.3 Platform coverage asymmetry

The same brand's citation rate varies dramatically across AI platforms. Aggregated across our 5 pilots, relative strength:

Fig 11-2: Platform coverage asymmetry (anonymized)

```
flowchart LR
    subgraph HighLang["English-strong brands (Brand A/B/C/E)"]
      H1[ChatGPT  ]
      H2[Claude  ]
      H3[Perplexity  ]
      H4[DeepSeek  ]
      H5[Kimi  ]
    end
    subgraph LocalLang["Chinese-local brand (Brand D)"]
      L1[ChatGPT  ]
      L2[Perplexity  ]
      L3[AI Overview  ]
      L4[DeepSeek  ]
      L5[Kimi  ]
    end
```

Fig 11-2: ☐☐ = materially cited; ☐ = mentioned; ☐ = near-zero. English-language B2B brands perform on US-origin AI; Chinese-local brands perform on Chinese models and Google AI Overview.

Takeaways

- **Do not over-invest on platforms that are not your battlefield** — Brand A had persistent zero citation rate on DeepSeek but solid citations on ChatGPT. Doubling down on DeepSeek would have been waste. Better to deepen the English-market ChatGPT presence.
- **Local-language brands should not ignore Chinese models** — Brand D initially assumed ChatGPT was the main battlefield, but Kimi and DeepSeek actually performed better for Taiwanese Chinese-language user queries. That was the real battlefield.
- **AI Overview's importance to physical businesses is under-rated** — much of Brand D's reach was from Google AI Overview (rather than from standalone AI apps), tightly coupled to GBP quality.

11.4 Schema.org completeness and citation rate

Fig 11-3: Completeness × citation-rate delta (aggregated)

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Completion (%) vs citation-rate delta after 6 weeks (illustrative)"
    x-axis ["< 40%", "40-60%", "60-80%", "80-100%"]
    y-axis "Citation rate delta (relative)" 0 --> 100
    bar [10, 25, 55, 80]
```

Fig 11-3: Citation-rate improvement over 6 weeks by completeness bucket, shown in relative terms. Brands above 80% completeness saw the largest citation gains.

Observations

- **Completeness correlates positively with citation-rate improvement** — but **not linearly**: brands below 60% progressed slowly; once past 60%, improvement accelerated
- This suggests a **"recognizability threshold"** — the AI may need a certain critical mass of structured facts before it begins proactively citing the entity
- **Beyond 80%, marginal returns decrease** — pushing from 85% to 100% does not deliver proportional citation gains

Operational takeaways

- **Prioritize getting every brand past 60%** — more efficient than pushing a few brands to 100%
- The customer anxiety around *"chasing 100%"* is largely wasted — 80% completeness plus **content quality** beats 100% mechanical fill
- Completeness is not the goal — it is **the means of being recognized by AI**. Once the recognition threshold is passed, move on to other levers.

11.5 AXP deployment before/after

Among the five pilots, three (A / B / C) only deployed AXP in Weeks 2–3, while two (D / E) had it active from Week 1. The time lag lets us observe the isolated effect of AXP.

Fig 11-4: AI bot traffic before/after AXP (anonymized aggregate)

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "AI bot daily visits (before vs after AXP, relative index)"
    x-axis ["Wk 1", "Wk 2", "Wk 3 (AXP deploy)", "Wk 4", "Wk 5", "Wk 6"]
```

```
    y-axis "AI bot visits (indexed)" 0 --> 200
    bar [25, 30, 35, 95, 140, 165]
```

Fig 11-4: AI bot traffic climbs rapidly in the week AXP is deployed. Index 100 = pre-deployment average.

Observations

- **AI bot traffic increased 3–5× within 2 weeks of AXP deployment** (mostly from GPTBot, ClaudeBot, PerplexityBot)
- **Citation-rate improvement lagged traffic by ~2–3 weeks** — once AI bots ingested content, another corpus-integration cycle had to pass before answers changed
- **GSC indexing also increased synchronously** — an SEO-side bonus; AXP's clean HTML is also friendly to traditional search

But — bot traffic rising does not guarantee citation rate rising. If the AXP content itself is semantically thin, the AI has nothing to work with. AXP is **infrastructure for being seen**, not a silver bullet.

11.6 Customer-side pitfalls

The customer-side operations produced five common pitfalls.

Fig 11-5: Pitfall distribution (aggregated across 5 brands, 6 weeks)

```
pie title "Customer-side pitfalls (5 brands, 6 weeks)"
    "industry_code not filled" : 28
    "logo_url missing or broken" : 22
    "GBP not connected (physical)" : 18
    "description too short (<20 chars)" : 17
    "sameAs external links unfilled" : 15
```

Fig 11-5: Missing industry classification is the most common pitfall; affects Schema.org `@type` selection.

Shared root causes

- **Customers do not know which fields matter most to AI** — without UI priority signals, customers fill in screen order and miss high-weight fields
- **Broken logo_url is more common than expected** — customers often paste internal paths, ephemeral CDNs, or image hosts that have moved; AXP generation then returns 404
- **Physical businesses aren't used to manually connecting GBP** — Brand D (our only physical pilot) took 3 weeks to complete GBP linking; half that time was lost to Place ID misunderstanding

UI feedback loop

Based on these observations, we made product changes:

- Completeness banner now **highlights "missing high-weight fields"** rather than showing all missing fields equally
- The `logo_url` field added **live validation** (HEAD request verifying 200 when pasted)
- Physical-business GBP connection now includes a **screenshot-annotated step-by-step guide**

11.7 Three unexpected findings

Three observations we did not anticipate but are worth recording.

1. Bilingual brands are surprisingly fragile on cross-language consistency

Brands A / C / E are all bilingual (zh / en). The AI's description of the *same brand* in Chinese vs English queries was **often materially different**. For example:

- English query: *"Acme is a B2B marketing-automation platform"*
- Chinese query: □Acme □□□□□□□□□□□□□(*"Acme is a marketing consulting service company"*)

This is not hallucination — both descriptions are partially true. It is that **AI's ability to aggregate the same entity across languages is still immature**. The takeaway for brand owners: Chinese and English Schema.org records should **explicitly link each other via** `sameAs`, and the descriptions should be semantically equivalent rather than independently written.

2. "No citation" is a harder problem than "negative citation"

We expected negative AI statements to be the biggest threat. The operational reality is the opposite: **"AI does not mention the brand at all"** is a worse problem. A negative mention at least proves the AI knows the entity exists and can be corrected via ClaimReview. Complete absence means **the brand is not in the candidate pool** — there is no handle to grab.

This explains why we set Citation Rate weight at 25% (see Ch 3) rather than higher — the metric matters, but if it dominates the total, other dimensions get marginalized.

3. Competitor co-occurrence can be a positive signal

Conventional wisdom: *"competitors showing up in the same AI answer dilutes your visibility."* We observed the opposite. **Being listed next to the right competitors reinforces the brand's category identity.** Brand A, early on, co-occurred with two well-known large competitors. While Citation Rate was modest, being bracketed at *"the same tier"* mentally by end-users turned into unusually strong downstream conversion.

More data needed to confirm this. But it suggests GEO's notion of *"friend-vs-foe"* may run opposite to traditional SEO's *"competitors."* In the AI era, **being placed alongside the right brands may matter more than being named alone**.

11.8 First-month commercial validation

The five brands above were a mix of internal dogfooding and partner pilots. In parallel, **Baiyuan GEO as a paid SaaS closed three commercial customers within its first paid month**, covering industries unlike the pilots:

Industry	Shape	Primary driver
Chain medical aesthetics	Multi-location physical, highly competitive category	GBP integration + physical LocalBusiness Schema.org + medical-grade hallucination detection
Emerging chain restaurant	Multi-location physical, rapid expansion	Location-level AXP + Phase baseline to capture expansion-period change
Premium aromatherapy / yoga	High ACV, word-of-mouth driven	Content Depth + Sentiment as the lead dimensions (narrative quality > citation frequency)

Observations

- **Three customers chose the platform for three distinct reasons** — validating the seven-dimension scoring hypothesis: industries prioritize different dimensions; no single metric serves the whole market
- **Chain aesthetics and chain restaurant share a "multi-location" requirement** — a business-side push for Phase 4 Organization Account (see §8.7), earlier than originally planned
- **Premium aromatherapy / yoga elevated Content Depth and Sentiment from secondary to primary** — high-ACV industries have longer decision chains; narrative quality beats frequency

These customers are fresh at time of writing; detailed operational data will appear in a future revision. The point here: **the engineering design of Baiyuan GEO is validated not only internally but by paid external demand**.

Key takeaways

- 5 brands × 6 weeks = *observations*, not *conclusions*; sample size prohibits statistical inference
- All seven dimensions improved; Position moves before Citation; Consistency converges latest
- Platform coverage has language and industry bias — focus on *"your real battlefield"* rather than all platforms
- Schema.org completeness correlates positively with citation rate, non-linearly; 60% is a recognition threshold; >80% shows diminishing returns
- AXP deployment drives 3–5× bot traffic; citation-rate improvement lags 2–3 weeks; traffic ≠ citation rate
- Five customer-side pitfalls (industry code / logo / GBP / description / sameAs); UI feedback via "high-weight missing" emphasis
- Three unexpected findings: bilingual consistency fragile, "no citation" worse than "negative citation", right competitor co-occurrence can be advantage
- First commercial month: 3 signed customers (chain aesthetics, emerging chain restaurant, premium aromatherapy/yoga), each driven by different dimensions

References

- Ch 3 — Seven-Dimension Scoring Algorithm
- Ch 6 — AXP Shadow Document
- Ch 7 — Schema.org Phase 1
- Ch 9 — Closed-Loop Hallucination Remediation

Chapter 12 — Limitations, Open Problems, and Future Work

A tool that explicitly lists what it cannot do is more trustworthy than one that claims omniscience.

Table of Contents

- 12.1 What the platform cannot do
- 12.2 Unpredictability of AI model version shifts
- 12.3 Open research problems
- 12.4 Roadmap
- 12.5 An invitation to practitioners and researchers
- Key takeaways
- References

12.1 What the platform cannot do

Fig 12-1: Current coverage matrix

Capability	Coverage	Gap
Monitoring	Complete	Only the 15 supported AI platforms; custom-deployed or private LLMs cannot be reached
Scoring	Complete	Cross-industry comparisons are not meaningful; query-space remains subjective
Structured data	Complete	Multi-language Schema.org only in zh-TW + en; Japanese, Korean, Southeast Asian pending
Hallucination detection	Partial	Depends on knowledge-source quality; coverage drops when sources are sparse
Hallucination remediation	Partial	Stubborn hallucinations still need human intervention
Automated closed loop	Partial	Search-type converges quickly, knowledge-type slowly; intermediate states are hard to feedback fully

Capability	Coverage	Gap
External platform verification	Restricted	LinkedIn, Crunchbase, G2, Capterra have no public API; manual only
GBP integration	Restricted	Phase 2 API approval pending; only URL-to-Place-ID extraction available today

Fig 12-1: "Complete" = feature is comprehensive; "Partial" = core is there with known gaps; "Restricted" = blocked by external constraints.

Specific limits

- **Cannot 100% verify every external platform** — LinkedIn, Crunchbase, and similar without a public API can only be checked manually for presence; we cannot programmatically compare content accuracy
- **GEO score is not an absolute value** — query spaces differ across industries, making cross-industry number comparisons meaningless. Valid comparisons are only within the same industry, same time window, same query pool.
- **Chinese-language AI coverage lags English** — Chinese LLMs' depth of recognition for non-major brands is still noticeably below English models. We mitigate by strengthening the L1 LLM Wiki's Chinese fact structuring, but cannot fully close the gap.
- **No webhook mechanism** — most AI platforms do not emit *"brand was mentioned"* events; polling causes latency.

12.2 Unpredictability of AI model version shifts

This is a problem **we cannot fully solve from the engineering side**. When OpenAI releases GPT-5, Anthropic releases Claude 4, or DeepSeek ships a new flagship, every brand's score may shift 3–10 points simultaneously.

Three classes of version shift

Type	Example	Direction
Major model upgrade	GPT-4o ☐ GPT-5	Most brands rise (newer training data)
Safety / alignment tightening	One vendor increases refusal rate	Most brands fall (refusal masks citations)
Retrieval augmentation on/off	Claude adds or removes web search	Direction differs by brand based on web presence

Mitigation

Baiyuan cannot prevent these shifts, but three mechanisms reduce customer impact:

1. **Version-sensitivity banner** — when a major version change is detected on a tracked AI platform, the UI displays *"data is adapting to the new model; short-term volatility is expected."*

2. **Phase baseline cross-version tagging** — baseline data across model versions is explicitly **not comparable by raw numbers**; the UI distinguishes them
3. **Weight-preserved historical comparison** — internally we retain *"score under specific version"* for trend analysis, so version jumps are not misattributed to brand change

---

12.3 Open research problems

1. Real negative feedback vs hallucination error

When AI says *"this brand has poor customer service,"* it could be:

- **Hallucination** — AI associated the claim from an unrelated source
- **Real review** — actual negative user reviews have leaked into training corpora

The handling differs drastically: hallucination should be corrected; real feedback should drive service improvement, not concealment. Baiyuan's automation today **cannot reliably tell them apart**; human intervention is needed for source judgment. This is a real hole in the closed loop.

2. Causation vs correlation

A customer revises content; three weeks later, citation rate rises. Is this:

- **Causation** — the content revision directly improved AI recognition
- **Correlation** — another event (news, paid placement, seasonality) drove the improvement

Rigorous causal proof would require A/B-testing infrastructure (half of the same brand revised, half not) — commercially not feasible. This is a **shared research gap for the GEO field**.

3. Long-tail query coverage strategy

Dynamic intent-query generation covers the main intent types with 20–60 queries. But **long-tail queries** (very specific, uncommon user questions) cannot be enumerated. When a customer says *"my user asked XX and AI didn't mention me,"* is that:

- Random sampling missing a long-tail that was bound to be missed
- A systematic coverage gap

Currently handled case-by-case. A future *"customer-supplied intent queries"* feature could help, but would introduce *"customers only ask flattering questions"* bias.

12.4 Roadmap

Fig 12-2: Future work dependency graph

```
flowchart LR
    subgraph Short["Short-term (within 6 months)"]
      A1[GBP API Phase 2-3<br/>read and write]
      A2[multi-language Schema.org<br/>extend to ja / ko]
      A3[visualization upgrade<br/>Phase baseline views]
    end
    subgraph Mid["Mid-term (6-12 months)"]
      B1[more AI platforms<br/>Mistral / Cohere deepening<br/>+ Claude Projects]
      B2[cross-language sameAs<br/>automation]
      B3[competitor co-occurrence advisor]
    end
```

```
subgraph Long["Long-term (12+ months)"]
  C1[causal inference research<br/>A/B methodology]
  C2[private-LLM entity monitoring]
  C3[multi-tenant custom intent queries]
end
A1 --> B1
A2 --> B2
A3 --> C3
```

Fig 12-2: Three-phase roadmap. Each phase gates on the previous. Concrete timing depends on external factors (Google, specific AI vendors).

Short-term focus

- **GBP API Phases 2–3** — the single biggest pending item. Phase 2 (read) expected within 1–2 months of approval; Phase 3 (LocalPosts write) after Phase 2 has been stable for 3 months.
- **Multi-language Schema.org** — addresses Japanese and Korean market customer demand; language models re-use existing API, only i18n content and `@type` mappings need extending.
- **Phase baseline visualization** — most frequently requested customer UX item.

Long-term targets

- **Causal inference research** — partner with an academic institution to publish an A/B methodology for GEO, becoming shared foundation for the field.
- **Private-LLM monitoring** — serve enterprise customers' internal AI applications (support bots, internal knowledge assistants) for entity-recognition auditing — a product line adjacent to the current SaaS.

12.5 An invitation to practitioners and researchers

This book attempts to make GEO *a discipline that can be discussed and advanced collectively*, rather than the closed experience of a single vendor. To that end:

- **Please cite, rewrite, translate** any chapter (per the CC BY-NC 4.0 license)
- **Please file errata, questions, additions** via GitHub Issues (templates provided)
- **Collaborative research proposals welcome** — especially on causal inference, cross-language entity alignment, and stubborn hallucinations
- **Reuse architectural patterns** — Stale Carry-Forward, multi-provider fault tolerance, central shared RAG, closed-loop remediation are general-purpose engineering designs

GEO is very early. This book aspires to be *one of the first openly published technical documents* in this field, so that later teams can start from the holes we already crawled out of rather than rediscovering each one independently.

Key takeaways

- Baiyuan GEO has explicit limits: external platform verification restricted, Chinese-model coverage lagging, no webhook mechanism
- AI model version shifts are unpredictable; mitigation via banners, baseline slicing, weight-preserved comparison
- Three open problems: real-negative vs hallucination, causation vs correlation, long-tail query coverage
- Roadmap partitioned short (GBP API completion) / mid (cross-language sameAs) / long (causal inference research)

- Invitation to practitioners: cite the book, discuss on GitHub, propose research, reuse architecture

References

- Ch 3 — Seven-Dimension Scoring (score limits discussion)
- Ch 8 — GBP API Phase Roadmap
- Ch 9 — Closed-Loop Remediation (boundaries of automation)
- Ch 11 — Field Observations and Unexpected Findings
- Repository README — contribution and citation

Appendix A — Glossary (Full)

Alphabetical. Each entry provides: **definition + first chapter mention + related chapters**.

A

- **@id (Schema.org)** — JSON-LD unique identifier string used to reference entities within `@graph` or across documents. First in: §6.7. Core use: §7.3.
- **AI Bot** — umbrella term for crawlers that ingest web content for LLM training or retrieval (GPTBot, ClaudeBot, PerplexityBot, etc.). First in: §6.3.
- **AI Citation Rate** — fraction of representative intent queries in which the brand is proactively mentioned by an AI (0–100%). The book's core metric. First in: §1.2. Deep dive: §3.2.1.
- **aggregateRating** — Schema.org property carrying aggregate review scores. Synced from GBP in our platform. §8.4.
- **AXP (AI-ready eXchange Page)** — Baiyuan coinage. A "shadow document" served to AI bots — pure HTML + Schema.org JSON-LD + Markdown, decoupled from the human-facing website. Full treatment: Ch 6.
- **App-level filter** — application-layer query-level filtering; in our multi-tenant design, the second line of defense alongside RLS. §2.5.

B

- **BullMQ** — Redis-backed Node.js job-queue library; the backbone of our background-task infrastructure. §2.4.
- **Brand Entity** — umbrella term for structured brand data rooted at a Schema.org Organization / LocalBusiness node. Ch 7.

C

- **ChainPoll** — a verification method in which the same claim is sent to the LLM 3 times with the same prompt and the majority result is taken. Used in the NLI confidence 0.5–0.8 uncertain band. §9.3.
- **CID (Customer ID)** — Google Maps internal numeric identifier; cannot be converted to Place ID without a Places API call. §7.7.
- **ClaimReview** — a Schema.org property for annotating whether a claim is true. Used by the platform to publicly declare a hallucination has been corrected. §9.6.
- **Closed-Loop** — the detect ☐ compare ☐ decide ☐ generate ClaimReview ☐ inject AXP/RAG ☐ rescan cycle. Ch 9.
- **Cloudflare Workers** — edge-compute runtime used for AI-bot UA detection and AXP injection. §6.3.
- **Common Crawl** — publicly available large-scale web scrape; a primary pretraining source for most major LLMs. §1.3.
- **Consistency** — one of the seven dimensions; inverse of cross-platform standard deviation of the other six dimensions. §3.2.7.
- **Content Depth** — one of the seven dimensions; the depth of the description that accompanies a brand mention. §3.2.6.
- **Core Web Vitals** — Google's page-experience metrics (LCP, FID, CLS). Material for SEO; barely relevant in GEO. §1.3.

D

- **DIRECT_MODEL_ID_MAP** — translation table between aggregator model IDs and vendor-native model IDs. §5.3.1.
- **Dogfooding** — using one's own product internally. Brand E in our case studies is this scenario. §11.1.

E

- **entailment / contradiction / neutral / opinion** — the four NLI classification outputs. Only `contradiction` is flagged as a hallucination; `neutral` is explicitly *not* one. §9.3.
- **extraParams** — vendor-specific required API parameters, e.g., Qwen3 requires `enable_thinking: false`. §5.3.2.

F

- **Fingerprint** — hash of an AXP or Schema.org artifact, used to determine whether AI has ingested updated content. Ch 9 related.
- **FTID (Feature ID)** — Google Maps internal location identifier. §7.7.

G

- **GBP (Google Business Profile)** — Google's business listing; source of truth for physical businesses. Ch 8.
- **GEO (Generative Engine Optimization)** — the discipline and practice of improving a brand's mention frequency, position, and narrative accuracy inside generative-AI responses. Ch 1.
- **GPTBot** — OpenAI's web crawler for ChatGPT training-corpus collection. §6.4.
- **Ground Truth (GT)** — factual ground truth. In this book: the authoritative reference set used to compare candidate claims (live-scraped website / RAG / manual GT — three tiers). §9.3.

H

- **Hallucination** — AI-generated content that does not match reality. Split in this book into 5 types: entity misidentification, business fact error, industry miscategory, temporal/spatial error, attribute error. §9.2.

I

- **industry_code** — 25-industry classification enum used by the platform. §7.2.
- **Intent Query** — a representative user question generated dynamically based on a brand's industry. §2.2.1.
- **is_physical** — flag on a brand entity indicating physical vs online business; drives Schema.org field weights. §7.4.
- **isStale** — marker set under Stale Carry-Forward indicating a platform's score is a historical carried-forward value. §4.3.

J

- **JSON-LD** — JavaScript Object Notation for Linked Data; the dominant serialization format for Schema.org. §6.2, Ch 7.

K

- **kgmid (Knowledge Graph Machine ID)** — Google Knowledge Graph entity identifier. Used in `sameAs`.
- **Knowledge Graph** — structured network of entities and relationships. Built via `@id` interlinking in Schema.org. §7.3.

L

- **Layer 1 sentinel scan** — 4-hour cadence lightweight verification against search-type AI. §9.7.

- **Layer 2 full scan** — 24-hour cadence deep scan with seven-dimension scoring + fingerprint comparison. §9.7.
- **LLM Wiki** — Baiyuan coinage. An active semantic-knowledge layer maintained by LLMs, sitting at L1 inside the RAG architecture. §9.5.
- **LocalBusiness** — Schema.org `@type` commonly used for physical businesses. §7.2.

M

- **modelRouter** — in-house multi-AI-provider routing service providing a primary/fallback dual-path abstraction. Ch 5.

N

- **NLI (Natural Language Inference)** — the task of classifying the entailment relationship between two passages. Our primary hallucination-detection mechanism. §9.3.

O

- **OpenAI-compatible API** — aggregator endpoints that wrap multiple AI vendors behind OpenAI's SDK interface. §5.2.

P

- **pgvector** — PostgreSQL extension for vector embeddings and similarity search. §2.4.
- **Phase Baseline Testing** — longitudinal test using a fixed 20-question set re-asked across different time points. Ch 10.
- **Place ID** — Google Maps unique location identifier; primary key of the Places API. §7.7.
- **Position Quality** — one of the seven dimensions; weighted average of positions at which the brand appears in AI responses. §3.2.2.
- **Platform Breadth** — one of the seven dimensions; platforms with at least one mention ÷ total monitored platforms. §3.2.4.

Q

- **Query Coverage** — one of the seven dimensions; diversity of intent-query types in which the brand is mentioned. §3.2.3.
- **QPM (Queries Per Minute)** — API quota unit. GBP API default is 300 QPM. §8.5.

R

- **RAG (Retrieval-Augmented Generation)** — generation augmented by retrieval. Our platform uses a centralized shared RAG SaaS architecture. §9.4.
- **RLS (Row-Level Security)** — PostgreSQL row-level access control; first line of defense in our multi-tenant isolation. §2.5.

S

- **sameAs** — Schema.org property pointing to the same entity on other platforms (Wikipedia, Wikidata, LinkedIn, GBP). §7.3.
- **Schema.org** — the structured-data vocabulary jointly promoted by Google, Bing, Yahoo, and Yandex. Ch 7.
- **Sentiment Score** — one of the seven dimensions; aggregated tone of mentions (positive / neutral / negative). §3.2.5.
- **Sentinel Scan** — see Layer 1.

- **SERP (Search Engine Results Page)** — the core battlefield of classical SEO. §1.1.
- **Sitemap** — XML URL list that helps crawlers discover content. §6.6.
- **Smoke Test** — minimal validation run on fallback paths at service startup, to ensure they are actually reachable. §5.6.1.
- **Stale Carry-Forward** — Baiyuan coinage. When a platform's scan fails 100%, carry forward the last successful value from history and mark it as stale; used to maintain signal continuity. Ch 4.

T

- **Tenant ID (X-Tenant-ID)** — multi-tenant HTTP header used for tenant filtering in the central shared RAG. §9.4.

U

- **UA (User-Agent)** — HTTP request header; the basis on which AXP injection decides AI-bot vs human. §6.4.

W

- **Webhook** — server-initiated event notification. GBP API **does not provide** webhooks. §8.6.
- **Wikidata** — open knowledge graph; a key target for Schema.org `sameAs`. §7.3.
- **Wizard** — linear seven-step Schema.org onboarding flow for new brands. §7.6.

Standards cross-reference

Standard	Role	Chapters
Schema.org	structured-data vocabulary	Ch 6, 7, 9
JSON-LD 1.1	Linked Data serialization	Ch 6, 7
OpenGraph	meta tags for link previews	Ch 6
robots.txt	Robots Exclusion Protocol	Ch 6
sitemaps.xml	Sitemap Protocol 0.9	§6.6

Appendix B — Public API Specification (Excerpt)

This appendix excerpts the **public-facing** REST endpoints of Baiyuan GEO Platform, for third-party integration, research, and audit reference. Internal admin endpoints (admin, monitoring, billing) are not included.

Common conventions

Base URL

`https://api.geo.baiyuan.io/v1`

Authentication

All endpoints require an `Authorization: Bearer <token>` header. Tokens are obtained from the customer panel's *"API key management"* page.

Response envelope

Every response has the form:

```
{
  "status": "success" | "error",
  "data":   { ... },           // when status=success
  "error":  "description"      // when status=error
}
```

Rate limit

- Default 60 req/min; raisable on enterprise plans
- Exceeded ☐ HTTP 429 with `Retry-After` header indicating wait seconds

---

1. Brands

GET `/brands`

List brands visible to the current account.

Query params:

- `limit` (1–100, default 20)
- `offset` (default 0)
- `industry_code` (optional; see the 25-category enum in §7.2)

Response:

```
{
  "status": "success",
  "data": {
    "brands": [
      {
        "id": "brd_01H...",
        "name": "Example Brand",
        "industry_code": "saas_application",
        "url": "https://example.com",
        "completion": 78,
        "current_geo_score": 62
```

```
      }
    ],
    "total": 42,
    "limit": 20,
    "offset": 0
  }
}
```

GET `/brands/:id`

Get a single brand's details.

POST `/brands`

Create a new brand. **Body**: `name` (required), `url`, `industry_code`, `description`, etc.

PUT `/brands/:id`

Update brand. Body semantics depend on `Content-Type`; merge-patch supported.

---

2. Scans

POST `/scans`

Trigger a scan immediately (not scheduled).

Body:

```
{
  "brand_id": "brd_01H...",
  "platforms": ["chatgpt", "claude", "perplexity"],
  "intent_count": 20,
  "scan_layer": "full"
}
```

Response (202 Accepted):

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
    "estimated_duration_seconds": 180,
    "queue_position": 3
  }
}
```

GET `/scans/:id`

Query scan status.

Response:

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
```

```
    "state": "queued" | "running" | "completed" | "failed",
    "progress": 0.75,
    "started_at": "2026-04-18T09:15:00Z",
    "completed_at": null,
    "results_url": "/scans/scn_01H.../results"
  }
}
```

GET `/scans/:id/results`

Get scan results.

Response (abbreviated):

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
    "scan_layer": "full",
    "geo_score": 62,
    "dimensions": {
      "citation":    { "value": 55, "isStale": false },
      "position":    { "value": 70, "isStale": false },
      "coverage":    { "value": 60, "isStale": false },
      "breadth":     { "value": 58, "isStale": false },
      "sentiment":   { "value": 72, "isStale": false },
      "depth":       { "value": 50, "isStale": false },
      "consistency": { "value": 65, "isStale": false }
    },
    "platforms": {
      "chatgpt":    { "sov": 60, "position": "top-third" },
      "claude":     { "sov": 58, "position": "top-third" },
      "perplexity": { "sov": 48, "position": "middle" }
    }
  }
}
```

---

3. Entity (Schema.org brand entity)

GET `/brands/:id/entity`

Get a brand's complete Schema.org data (Organization, Services, Employees, Hours, Location, Social Links).

GET `/brands/:id/entity/schema.json`

Get the generated Schema.org JSON-LD (embeddable directly into the customer's website).

Response: `Content-Type: application/ld+json`; body is `{"@context": "https://schema.org", "@graph": [...]}`.

GET `/brands/:id/entity/completion`

Get completeness score.

Response:

```
{
  "status": "success",
  "data": {
    "total_percent": 78,
    "breakdown": {
      "basic_info":  { "percent": 100, "weight_pct": 25 },
      "location":    { "percent":  80, "weight_pct": 15 },
      "services":    { "percent":  60, "weight_pct": 15 },
      "employees":   { "percent":  40, "weight_pct": 10 }
    },
    "missing_high_priority": ["description", "logo_url"]
  }
}
```

---

4. AXP (shadow document)

GET `/axp?host=<domain>&path=<path>`

This endpoint is **called by the Cloudflare Worker**; integrators do not need to call it directly. Returns the shadow document HTML for the given brand and path.

---

5. Hallucinations

GET `/brands/:id/hallucinations`

List detected hallucinations.

Query params: `severity` (critical/major/minor/info), `status` (open/resolving/resolved).

POST `/brands/:id/hallucinations/:hid/repair`

Trigger remediation flow.

---

6. Baseline (Phase baseline testing)

POST `/brands/:id/baseline`

Create a new Phase baseline cohort.

POST `/brands/:id/baseline/:cohortId/run`

Re-run an existing baseline.

GET `/brands/:id/baseline/:cohortId/compare`

Get Phase 1 vs Phase 2 / 3 comparison.

---

7. Webhooks (customer-receivable)

Baiyuan platform **pushes** the following events to customer-configured webhook URLs (set in the panel):

Event	Trigger
`scan.completed`	A scan finished
`hallucination.detected`	A new hallucination detected (severity >= major)
`hallucination.resolved`	A hallucination converged
`completion.threshold_crossed`	Completeness crossed a 60% / 80% / 100% threshold
`gbp.sync_failure`	GBP sync failed 3 times consecutively

Payload shape:

```
{
  "event": "scan.completed",
  "brand_id": "brd_01H...",
  "scan_id": "scn_01H...",
  "occurred_at": "2026-04-18T09:30:00Z",
  "data": { ... }
}
```

Every webhook request carries an `X-Baiyuan-Signature` header (HMAC-SHA256); customers should verify before processing.

Error codes

HTTP	Code	Description
400	`invalid_parameter`	Parameter format or value invalid
401	`unauthenticated`	No token or token invalid
403	`forbidden`	Token valid but no resource access
404	`not_found`	Resource does not exist
409	`conflict`	State conflict (e.g., scan in progress, cannot re-trigger)
422	`unprocessable_entity`	Data validation failed
429	`rate_limited`	Quota exceeded
500	`internal_error`	Internal error — retry or contact support

Appendix C — References

Every external source cited in this book. Grouped into four categories.

1. Official specifications and standards

Schema.org and Linked Data

- Schema.org. *Schema.org Vocabulary Specification*. https://schema.org/docs/schemas.html
- W3C. *JSON-LD 1.1: A JSON-based Serialization for Linked Data*. https://www.w3.org/TR/json-ld11/
- Schema.org. *ClaimReview*. https://schema.org/ClaimReview
- Schema.org. *LocalBusiness*. https://schema.org/LocalBusiness
- Schema.org. *Organization*. https://schema.org/Organization
- Schema.org. *Service*. https://schema.org/Service
- Schema.org. *Person*. https://schema.org/Person

Google ecosystem

- Google Business Profile API. *Overview*. https://developers.google.com/my-business
- Google Business Profile API. *Notifications Overview*. https://developers.google.com/my-business/content/notification-setup
- Google Maps Platform. *Places API documentation*. https://developers.google.com/maps/documentation/places/web-service
- Google Search Central. *How Search works*. https://www.google.com/search/howsearchworks/
- Google Search Central. *Fact check article markup*. https://developers.google.com/search/docs/appearance/structured-data/factcheck
- Google Search Central. *Google-Extended and generative AI training*. https://developers.google.com/search/docs/crawling-indexing/overview-google-crawlers

AI bot User Agents

- OpenAI. *GPTBot User Agent Documentation*. https://platform.openai.com/docs/gptbot
- OpenAI. *OAI-SearchBot User Agent*. https://platform.openai.com/docs/bots
- Anthropic. *ClaudeBot documentation*. https://support.anthropic.com/en/articles/8896518
- Perplexity. *PerplexityBot*. https://docs.perplexity.ai/guides/bots
- Common Crawl. *CCBot User Agent*. https://commoncrawl.org/ccbot

Web protocols

- IETF. *RFC 9309 — Robots Exclusion Protocol*. https://www.rfc-editor.org/rfc/rfc9309.html
- sitemaps.org. *Sitemap Protocol 0.9*. https://www.sitemaps.org/protocol.html
- W3C. *HTTP/1.1*. https://www.rfc-editor.org/rfc/rfc7230

Open licenses

- Creative Commons. *CC BY-NC 4.0 Legal Code*. https://creativecommons.org/licenses/by-nc/4.0/legalcode
- Citation File Format. *CFF 1.2.0 Specification*. https://github.com/citation-file-format/citation-file-format

---

2. Technical frameworks and development tools

- Cloudflare. *Workers Runtime Documentation*. https://developers.cloudflare.com/workers/

- Next.js. *App Router & Server Components*. https://nextjs.org/docs
- PostgreSQL. *Row Security Policies*. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- pgvector. *Open-source vector similarity search for Postgres*. https://github.com/pgvector/pgvector
- BullMQ. *Queue documentation*. https://docs.bullmq.io/
- OpenAI. *Node.js SDK*. https://github.com/openai/openai-node
- Anthropic. *TypeScript SDK*. https://github.com/anthropics/anthropic-sdk-typescript

---

3. Industry research and statistics

- OpenAI. (2025). *Usage & Revenue Update, Q3 2025*. Official quarterly disclosure. (Cited in Ch 1)
- Perplexity AI. (2025). *Year in Review 2024: Search Volume & Engagement*. Official blog. (Cited in Ch 1)
- SimilarWeb. (2025). *The State of Generative Search: AI Overview Impact on Publisher Traffic*. Research report. (Cited in Ch 1)
- Google. (2024). *Generative AI in Search: Let Google do the searching for you*. Official announcement.

---

4. Books and monographs

- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media. ISBN 978-1449373320.
  - Chapter 8, *"The Trouble with Distributed Systems,"* is the general reference framework for Ch 4's Stale Carry-Forward pattern.
- Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf. ISBN 978-1680502398.
  - Circuit Breaker, Bulkhead, and Timeout patterns are the design references for Ch 5's modelRouter.
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*. IT Revolution. ISBN 978-1942788003.
  - The closed-feedback design of Ch 9 is informed by the *"continuous learning"* chapter of the Third Way.

---

5. Related academic work (selected)

This book did not perform a comprehensive literature review. The papers below are representative works the author has read; readers are welcome to suggest additions via GitHub Issues.

LLM hallucination detection

- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models*. EMNLP 2023.
- Min, S., Krishna, K., Lyu, X., et al. (2023). *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. EMNLP 2023.

Knowledge graphs and entity linking

- Pan, J. Z., Razniewski, S., Kalo, J.-C., et al. (2024). *Large Language Models and Knowledge Graphs: Opportunities and Challenges*. TGDK 2024.
- Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). *Knowledge Graphs*. ACM Computing Surveys, 54(4).

Multi-source NLI

- Nie, Y., Chen, H., & Bansal, M. (2019). *Combining Fact Extraction and Verification with Neural Semantic Matching Networks*. AAAI 2019.
- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). *FEVER: A Large-scale Dataset for Fact Extraction and VERification*. NAACL 2018.

Structured data and LLMs

- Yang, J., Chen, H., Yan, Y., et al. (2024). *Exploring the Role of Structured Data in Large Language Model Fact Verification*. Preprint.

How to cite this book

The canonical citation formats are in `CITATION.cff` and in the citation section of the README.

```
@techreport{lin2026baiyuangeo,
  author      = {Lin, Vincent},
  title       = {Baiyuan GEO Platform: A Whitepaper on Building a SaaS for Generative Engine Optimizatio
  institution = {Baiyuan Technology},
  year        = {2026},
  url         = {https://github.com/baiyuan-tech/geo-whitepaper},
  note        = {v1.0-draft}
}
```

Appendix D — Figure Catalog and Production Guidelines

All figures across the English edition are indexed here. Mermaid diagrams embedded directly in `.md` files render natively on GitHub; other static images (if any) live in `assets/figures/`.

D.1 Complete catalog

ID	Type	Subject	Chapter
Fig 1-1	xychart-beta	2023–2025 information-channel migration	Ch 1
Fig 1-2	table	SEO vs GEO core differences	Ch 1 §1.3
Fig 2-1	flowchart	Six-stage closed-loop system	Ch 2 §2.1
Fig 2-2	flowchart	Full brand governance cycle	Ch 2 §2.3
Fig 2-3	flowchart	Multi-tenant RLS + app-level double insurance	Ch 2 §2.5
Fig 3-1	graph	Seven-dimension radar adjacency	Ch 3 §3.2
Fig 4-1	xychart-beta	Three strategies under platform outage	Ch 4 §4.3
Fig 4-2	stateDiagram	Platform-data state machine (Fresh/Stale/Expired/Reset)	Ch 4 §4.3
Fig 4-3	ASCII	Frontend isStale badge	Ch 4 §4.6
Fig 5-1	flowchart	modelRouter dual-path architecture	Ch 5 §5.2
Fig 5-2	flowchart	Switch decision tree	Ch 5 §5.5
Fig 6-1	flowchart	Two views of the same brand	Ch 6 §6.1
Fig 6-2	flowchart	AXP three-layer structure	Ch 6 §6.2
Fig 6-3	flowchart	Worker routing decision flow	Ch 6 §6.3
Fig 6-4	flowchart	25 AI bot UAs in four groups	Ch 6 §6.4
Fig 6-5	flowchart	SaaS self-brand path-category decision tree	Ch 6 §6.5
Fig 6-6	code block	JSON-LD flat vs nested-array	Ch 6 §6.7
Fig 7-1	table	25-industry classification table	Ch 7 §7.2
Fig 7-2	flowchart	Three-layer entity knowledge graph	Ch 7 §7.3
Fig 7-3	flowchart	Physical vs online weight divergence	Ch 7 §7.4
Fig 7-4	flowchart	Wizard + Edit dual entry points	Ch 7 §7.6
Fig 7-5	flowchart	GBP URL parser four-branch decision tree	Ch 7 §7.7
Fig 8-1	flowchart	One-way data flow: GBP ☐ Schema ☐ AXP ☐ AI	Ch 8 §8.1
Fig 8-2	flowchart	Two hosting models (Manager / OAuth)	Ch 8 §8.3
Fig 8-3	table	Sync frequency matrix	Ch 8 §8.5
Fig 8-4	flowchart	GBP integration four-phase roadmap	Ch 8 §8.7
Fig 9-1	flowchart	Six-stage closed-loop	Ch 9 §9.1
Fig 9-2	flowchart	Hallucination five-type taxonomy	Ch 9 §9.2
Fig 9-3	flowchart	Three-tier knowledge sources into NLI	Ch 9 §9.3

ID	Type	Subject	Chapter
Fig 9-4	flowchart	Central shared RAG architecture	Ch 9 §9.4
Fig 9-5	flowchart	LLM Wiki document lifecycle	Ch 9 §9.5
Fig 9-6	flowchart	ClaimReview three injection paths	Ch 9 §9.6
Fig 9-7	flowchart	Two-tier scan division of labor	Ch 9 §9.7
Fig 9-8	xychart-beta	Hallucination prevalence convergence curve	Ch 9 §9.8
Fig 10-1	flowchart	Regular scan vs baseline	Ch 10 §10.1
Fig 10-2	flowchart	Phase 1/2/3 data structure	Ch 10 §10.2
Fig 10-3	flowchart	Four-axis change-observation matrix	Ch 10 §10.4
Fig 11-1	graph	5-brand seven-dimension radar (Wk 1 vs Wk 6)	Ch 11 §11.2
Fig 11-2	flowchart	Platform coverage asymmetry (English vs local)	Ch 11 §11.3
Fig 11-3	xychart-beta	Completeness × citation-rate delta	Ch 11 §11.4
Fig 11-4	xychart-beta	AI bot traffic before/after AXP	Ch 11 §11.5
Fig 11-5	pie	Customer-side pitfall distribution	Ch 11 §11.6
Fig 12-1	table	Current coverage matrix	Ch 12 §12.1
Fig 12-2	flowchart	Future work dependency (short/mid/long)	Ch 12 §12.4

Total: 13 chapters, 44 figures (Mermaid dominant, with a few ASCII art, tables, and code blocks).

D.2 Production style guide

1. Unified style

Use the following palette (apply via `%%{init: {'theme':'base'}}%%` + targeted `style` overrides):

Use	Hex
Primary (node borders, emphasis)	`#1e40af` (deep blue)
Accent (emphasis, arrows)	`#ea580c` (bright orange)
Warning (errors, failure paths)	`#dc2626` (red)
Success (fixes, convergence)	`#16a34a` (green)
Neutral (backgrounds, helpers)	`#64748b` (slate)

Mermaid `style` syntax can apply these to specific nodes:

```
flowchart LR
    A[node]
    style A fill:#1e40af,color:#fff
```

2. Prefer Mermaid over static images

- **Technical flows** (flowchart / stateDiagram / sequenceDiagram) — all Mermaid

- **Data relationships** (graph, ER) — Mermaid
- **Trend charts** (xychart-beta) — Mermaid
- **Pie charts** — Mermaid

Rationale:

- Version-controlled, diffable, and readers can copy and remix
- No external image-host dependency (image hosts can fail)
- AI crawlers can read the source Mermaid code; the figure itself is structured data

3. When to use static images (SVG/PNG)

Only in these cases, placed under `assets/figures/`:

- Complex visuals Mermaid cannot express (deeply nested layouts, illustrative artwork)
- Product screenshots (UI frames, customer-panel examples)
- Charts that need Recharts interactivity (become static PNG in PDF)

All static images must:

- Retain source files (`.fig`, `.drawio`, Figma link)
- Use SVG by default (scalable, small)
- Name as `fig-<chapter>-<number>-<slug>.svg`, e.g., `fig-11-03-completion-citation.svg`

4. Alt text

Every static image needs descriptive alt text:

```
![Fig X-Y subject: concrete content<br>*Fig X-Y: detailed caption*](assets/figures/fig-XX-YY-slug.svg)
```

Alt text should let a reader who cannot see the image understand the figure's point (serves visually impaired users and AI crawlers simultaneously).

5. Bilingual editions produced independently

- Chinese figures are **not translated** into English; the English edition redraws using English conventions
- Data units, date formats, and text direction adjusted per locale
- Both editions share figure numbers (Fig 1-1 Chinese / Fig 1-1 English in parallel)

6. De-identification of product screenshots

UI screenshots must:

- Redact customer brand names (use *"Brand A"*, *"Example Brand"*)
- Blur or replace real numbers (e.g., `$12,345` ☐ `$XX,XXX`)
- Redact internal URL paths (show structure, not concrete domain)
- Mask email addresses (`user@***.com`)

7. Mermaid in Pandoc PDF builds

Mermaid renders natively on GitHub Markdown, GitLab, Obsidian, and VS Code Preview. For **Pandoc PDF generation**, a preprocessing step is required:

- CI pre-renders code blocks to SVG via `mermaid-cli`
- Or use Pandoc filter `pandoc-mermaid` automatically

`build-pdf.yml` in this repo contains the placeholder-replacement approach used for v1.0-draft (see `.github/workflows/build-pdf.yml`).

D.3 Production status

Status	Figure count	Share
☐ Complete in `.md`	44	100%
☐ Need product screenshots	0	0%
☐ English edition redrawing	0 (pending en/ full edition)	—